

The Six-State Shuffle Bound

Three independent certificate computations,
one finite-to-infinite theorem,
constructive reachability

Research manuscript prepared for Vladimir Reshetnikov

20 September 2026

Abstract

The shuffle operation interleaves two words without changing the order of the letters within either word. Its exact worst-case deterministic state complexity is the subject of a published conjecture. This manuscript gives a computer-assisted proof of the following conjecture in the range $\min(m, n) \leq 6$.

Conjecture 0.1 (Published shuffle state-complexity conjecture). *For all $m, n \geq 2$, the worst-case deterministic state complexity of the shuffle is*

$$F(m, n) = 2^{mn-1} + 2^{(m-1)(n-1)}(2^{m-1} - 1)(2^{n-1} - 1). \quad (0.1)$$

This article proves that the bound (0.1) is attained whenever $m, n \geq 2$ and $\min(m, n) \leq 6$. The general conjecture is not settled here. The shared core theorem is proved once, from a single finite-to-infinite induction over the lexicographic rank $(m + n, |S|)$; the finite premise of that induction is then discharged three separate times, by three certificate databases that are genuinely independent of one another. Database A contains 10,642 spanning predecessor witnesses found by an SMT search and audited by an orbit-closure coverage enumeration. Database B contains 10,619 witnesses on a provably minimal, degree-sorted set of representatives; its target keys are a strict subset of A's, and not one of the 10,619 shared keys carries the same witness, so each database re-derives the other's finite lemma from scratch. Database C contains 453 witnesses on a strictly smaller irreducible class obtained by adding a fifth, analytic reduction — a balanced-triple contraction — which shrinks the finite obligation from widths at most twenty to widths at most eleven and replaces the solver by a deterministic permutation enumeration. The manuscript also proves three increasingly sharp access-word bounds, $|S| + \min(m, n) - 2$, the support form $E + \min(r, c) - 2$, and the dual bound $\min\{2(|S| - 1), |S| + \min(m, n) - 2\}$, and gives four distinguishability arguments, one of them over a fixed three-letter alphabet. The archive contains all three certificate databases, four independent checkers, four reaching-word constructors, replayable examples, and audit reports.

Status and scope. This is a computer-assisted research draft, not a peer-reviewed or proof-assistant-verified manuscript. No proof-assistant formalization and no independent human review is claimed for any part of it. The six-state result extends the range explicitly established in the primary sources located for this project; a targeted literature search found no prior resolution of this strip, but that negative search result is not a proof of novelty, and bibliographic priority remains provisional. No result for all $m, n \geq 7$, or for a fixed small alphabet, is claimed.

Contents

1	The problem and the result	4
1.1	A concrete question about interleaving languages	4
1.2	What is established here	4
1.3	Why this is a suitable finite attack	5
1.4	Provenance of this manuscript	5
1.5	Research status and novelty boundary	6
2	The universal shuffle automaton	6
2.1	From automata and grammars to a Boolean matrix	6
2.2	The full transformation-pair alphabet	7
2.3	Validity and the upper bound	7
3	Distinguishability requires no computation	9
3.1	A forward two-letter construction	9
3.2	The same statement by a cellwise existential predecessor	9
3.3	The same statement by enumerating run allocations	10
3.4	A fixed three-letter alphabet, and a stronger corollary	11
3.5	Minimality of the operands	12
4	Root-sensitive reductions	12
4.1	Relabeling and restriction to a subgrid	12
4.2	The one-dimensional base cases	12
4.3	Deleting a contained copy	13
4.4	A degree-one row or column gives an isolated edge	13
5	Hard matrices and spanning predecessor certificates	14
5.1	The irreducible targets	14
5.2	Why six rows need at most twenty columns	14
5.3	A root-independent local proof object	15
5.4	An exact inverse formulation	16
5.5	A maximal candidate when the letter is a pair of permutations	17
6	A fifth reduction: balanced triples	17
7	The finite-to-infinite theorem	18
7.1	The same induction with a weaker finite hypothesis	19
7.2	A sharper bound in terms of the support	20

7.3	A dual bound carried through the induction	21
7.4	Why this is not an extrapolation from finite experiments	21
8	The finite certificate computations	21
8.1	Encoding the targets	22
8.2	Database A: orbit-closure coverage	22
8.3	Database B: a minimal canonical representative set	27
8.4	Database C: residual cores and no solver at all	30
8.5	Why none of the three databases reduces to another	31
9	Completion of the state-complexity proof	32
10	Certificates that can be checked by hand	33
10.1	A six-by-seven target, certified from database A	33
10.2	The same target, certified from database B	34
10.3	An explicit reaching word	35
10.4	A different six-by-seven target, and a certificate no permutation search could find . .	35
10.5	A residual core, certified by a bare pair of transpositions	36
10.6	Two opposed remarks, and their resolution	37
10.7	Twenty columns and a width beyond the finite classification	38
11	Constructive algorithm and independent tests	38
11.1	Turning the proof into a word	38
11.2	Four constructors with different recursion discipline	39
11.3	Exact breadth-first checks on small grids	40
11.4	Replay, coverage, and negative tests	40
11.5	Three further regression suites	40
12	Reproduction and the trust boundary	41
12.1	What the archive contains	41
12.2	Verification without a solver	42
12.3	Three further verification paths	43
12.4	Optional regeneration of the search	44
12.5	Recorded environment and timing	44
12.6	Exactly what is, and is not, trusted	45
13	What remains open and how to extend the attack	45
13.1	The unrestricted conjecture	46
13.2	Three formulations of the open step	46

13.3 Alphabet size and the one-state exception	47
13.4 A connection with dependent proof objects	48
13.5 Four tempting shortcuts that would be invalid	49
14 Conclusion	49
A Certificate schemas and minimal verification cores	49
A.1 Coordinate-pair JSON lines, and its plain-integer twin	49
A.2 Fixed-width .cert records for database B	50
A.3 Bar-separated records for database C	51
B A review checklist	51
C Six-row antichains by number of columns	52
D The ten-letter word from database B	53
E The sixteen-letter word for the second six-by-seven target	54
F Detailed regression-test counts	55
G Proof dependency summary	55
References	55

1 The problem and the result

1.1 A concrete question about interleaving languages

Let $\kappa(L)$ denote the number of states in the minimal *complete* deterministic finite automaton recognizing a regular language L . For two words u and v , their shuffle is the set of all interleavings that preserve the internal order of each word. For instance,

$$ab \text{ shuf } c = \{abc, acb, cab\}.$$

The shuffle of two languages is obtained by taking the union over all choices of a word from each language. We write

$$\text{sc}_{\text{shuf}}(m, n) = \max\{\kappa(K \text{ shuf } L) : \kappa(K) = m, \kappa(L) = n\},$$

where the maximum ranges over pairs of languages over a common finite alphabet; the alphabet is allowed to depend on m and n .

The chosen open problem is the conjecture that, for every $m, n \geq 2$,

$$\text{sc}_{\text{shuf}}(m, n) = F(m, n) := 2^{mn-1} + 2^{(m-1)(n-1)}(2^{m-1} - 1)(2^{n-1} - 1). \quad (1.1)$$

Brzozowski, Jirásková, Liu, Rajasekaran, and Szykuła established the upper bound and its attainment for $2 \leq m \leq 5$ with arbitrary $n \geq 2$, as well as for $m = n = 6$ [1]. Their reduction to reachable subsets of a rectangular state space is the starting point here. Caron, Luque, and Patrou subsequently developed a partition-based reformulation and an antichain reduction, without settling the unrestricted conjecture [2]. These are the specific primary-source results against which this manuscript is compared.

1.2 What is established here

Theorem 1.1 (Six-state strip, computer-assisted). *For all integers $m, n \geq 2$ with $\min(m, n) \leq 6$,*

$$\text{sc}_{\text{shuf}}(m, n) = F(m, n).$$

In particular, for every $n \geq 2$,

$$\text{sc}_{\text{shuf}}(6, n) = 63 \cdot 2^{6n-6} - 31 \cdot 2^{5n-5}. \quad (1.2)$$

The part of this strip beyond the range explicitly obtained in the cited work is $m = 6, n \geq 7$, and its transpose. The present proof does not consist of checking a few additional rectangles. It reduces *all* widths to a finite collection of irreducible matrices and verifies a stronger, root-independent predecessor property for every member of that collection.

A quantitative reachability result accompanies the state-count theorem. For the universal alphabet introduced in Section 2, every valid subset S has a reaching word of length at most

$$|S| + \min(m, n) - 2. \quad (1.3)$$

Thus an exponentially large reachable subset automaton can nevertheless have short access words. The bound is not asserted to be optimal, and Section 7 in fact proves two strictly sharper statements alongside it: a *support form* $E + \min(r, c) - 2$, in which $E = |S|$ and r, c count the nonempty rows

and columns rather than the grid dimensions, and a *dual bound* $\min\{2(|S| - 1), |S| + \min(m, n) - 2\}$ whose first branch is independent of the dimensions altogether.

The finite premise of the induction is discharged three times over, by three certificate databases that are not reducible to one another (Section 8). Two of them classify the same irreducible remainder by two different symmetry-reduction correctness proofs and share no witness at all; the third classifies a strictly smaller remainder, obtained by an extra analytic reduction proved in Section 6. Each database alone suffices for Theorem 1.1. Keeping all three is what makes the finite step independently re-derivable rather than a single computation to be taken on trust.

1.3 Why this is a suitable finite attack

Two aspects make this problem particularly amenable to a certificate-based approach. First, a putative last letter is simply a pair of functions on finite sets. Second, the difficult matrices have incomparable columns. For six rows, an antichain contains at most $\binom{6}{3} = 20$ columns. This is a bound on the width of a *minimal obstruction*, not a restriction on the widths covered by the theorem.

The central methodological choice is to search for a predecessor that meets *every* row and every column. Such a predecessor is valid regardless of which row and column are distinguished as initial. This permits an unrooted classification, while retaining a sound rooted reachability induction. Without this precaution, quotienting by all row and column permutations can silently lose the initial-state condition.

1.4 Provenance of this manuscript

This document is a merge of four independently prepared research packages on the same conjecture. Every one of them proved the shared core theorem stated above; they are retained here because *how* each proved it differs, and those differences are mathematical content, not presentation.

- `shuffle-six-state-certificates` is the spine of the present text. It proved the theorem from database A: 10,642 spanning predecessor certificates over orbit-closure-checked representatives, discovered by an SMT search, validated by a Python local checker that exports a verified-target list, and audited for coverage by a separate C++ enumeration consuming only that list. It contributed the manuscript's structure, the tagged-multigraph reformulation, the Spanning Predecessor Conjecture, the certificate schema and minimal verification core, and the review checklist.
- `shuffle-six-state-strip` proved the theorem from the *same* database A by the same proof: its certificate file and the one shipped here agree in all 10,642 keys and all 10,642 witnesses, a fact recomputed during this merge. It is retained for what it adds around that shared core: the sharper support-form word bound with its explicit potential, a second plain-integer serialization with a record-by-record agreement test, a single-binary C++ verifier reporting totals across one through six rows, an iterative explicit-stack constructor, a 111,984-word regression suite, a run under undefined-behavior sanitization, and a standalone literature audit.
- `shuffle-six-state-theorem` proved the theorem from database B: an *independent* finite computation over a different canonicalization (degree-sorted representatives, proved to be exactly one per orbit), yielding 10,619 witnesses of which not one coincides with A's witness

for the same target. It also proved an additional structural property of its certificates and an entirely different distinguishability theorem over a fixed three-letter alphabet.

- **shuffle-six-state-complexity** proved the theorem by a materially different proof: a fifth analytic reduction removes an entire class of obstructions before any table is consulted, shrinking the finite obligation to database C — 453 witnesses over widths at most eleven — and replacing the solver by a deterministic, solver-free permutation enumeration whose output was regenerated byte for byte. It also proves the dual word-length bound and a fourth distinguishability argument.

Throughout, material carried over from a package other than the spine is integrated into the spine’s notation and numbering. Where two packages prove the same statement by genuinely different machinery, both proofs are kept and marked with a *parallel account* box saying what the variant buys.

1.5 Research status and novelty boundary

The source review was conducted on 20 September 2026. The accessible 2016 paper, the 2019 preprint of the later journal article, and the journal’s publication record were checked. The mathematical source review used the arXiv preprint of [2] rather than the access-restricted journal full text. Within [1] the located positions are the abstract; Table 1 on page 6 of the preprint, which marks the 6×7 and larger six-row cases as unresolved; Lemmas 8–9 on pages 7–8; Theorem 10 on page 8; and Lemma 12 with Corollary 13 on pages 9–10. In [2] the term “dense” states refers to incomparable row and column supports, not to graph density. Targeted searches did not identify a later resolution of the six-state strip or of the full conjecture; those searches were limited by the query formulations used and by indexing coverage, and did not reach unpublished or unindexed work.

This is evidence for choosing the problem, not a guarantee of an exhaustive literature search. Accordingly, the theorem is a mathematical claim supported by the proof and artifacts below, while its claim to bibliographic novelty remains provisional. The elementary reductions are presented with proofs rather than with claims of individual priority. The manuscript should receive independent mathematical and implementation review before being treated as an established research publication.

2 The universal shuffle automaton

2.1 From automata and grammars to a Boolean matrix

For a positive integer m , put $[m] = \{0, 1, \dots, m-1\}$. Let the state sets of two complete deterministic automata be $R = [m]$ and $C = [n]$. On a letter a , suppose their transitions are $f_a : R \rightarrow R$ and $g_a : C \rightarrow C$. The usual nondeterministic automaton for shuffle has state set $R \times C$ and transitions

$$(i, j) \xrightarrow{a} \{(f_a(i), j), (i, g_a(j))\}. \quad (2.1)$$

At each step, the current letter is assigned to exactly one of the two component words. A run records these assignments. Consequently a run reaching a pair of final states is equivalent to a decomposition of the input into an accepted word of the first automaton and an accepted word of the second, interleaved in their original orders. This proves that (2.1) recognizes shuffle.

There is a direct grammar interpretation. A complete deterministic automaton is presented as a deterministic right-linear grammar: introduce one nonterminal A_q for each state q , add a production $A_q \rightarrow aA_{\delta(q,a)}$ for each transition, and add $A_q \rightarrow \varepsilon$ at each accepting state; the start nonterminal corresponds to the initial state. Writing X_i for the nonterminal of the first operand's state i , its productions are $X_i \rightarrow aX_{f_a(i)}$ and, at final states, $X_i \rightarrow \varepsilon$. In the shuffle grammar, nonterminals $Z_{i,j}$ have the two productions

$$Z_{i,j} \rightarrow aZ_{f_a(i),j}, \quad Z_{i,j} \rightarrow aZ_{i,g_a(j)},$$

and an empty production at each pair of final states. Shuffle therefore combines two descriptions in a different way from concatenation: at each input letter one must decide which component produced it, and determinization tracks all component-state pairs compatible with those choices. The present question asks how large a deterministic finite-state description of this interleaving language may have to be. It therefore lies squarely in formal-language and regular-grammar theory, even though its remaining obstruction is combinatorial.

The measure being bounded below is *deterministic* state complexity. The theorem is not a lower bound of the same size on nondeterministic automata or on regular grammars, which can be much smaller, and it says nothing about the time needed to recognize a particular input word.

The determinized automaton has subsets $S \subseteq R \times C$ as states. We identify S with its m -by- n zero-one incidence matrix. A letter (f, g) acts by

$$\Delta_{f,g}(S) = \{(f(i), j) : (i, j) \in S\} \cup \{(i, g(j)) : (i, j) \in S\}. \quad (2.2)$$

The initial subset is a singleton $\{(r, c)\}$, where r and c are the initial states of the operands.

2.2 The full transformation-pair alphabet

Define

$$\Gamma_{m,n} = R^R \times C^C. \quad (2.3)$$

Each pair of total functions is a distinct letter. The alphabet is finite, with $m^m n^n$ letters. The two component automata read the same letter (f, g) by applying f and g , respectively.

Any pair of complete m - and n -state automata uses some selection of these transformation pairs, possibly with repeated pairs under different letter names. Thus allowing all pairs cannot remove any reachable subset. Conversely, the universal alphabet itself gives an admissible finite witness alphabet. Its use is therefore legitimate for a worst-case state-complexity theorem with no fixed alphabet bound. This universal model is also used in [1, Section 2.1].

In what follows, a “word” is a finite sequence of transformation pairs. Letters are not charged according to the size of their function tables; word length has its usual automata-theoretic meaning.

2.3 Validity and the upper bound

For $S \subseteq R \times C$, set

$$\text{rows}(S) = \{i : \exists j (i, j) \in S\}, \quad \text{cols}(S) = \{j : \exists i (i, j) \in S\}.$$

A subset is *valid relative to* (r, c) when

$$\text{Valid}_{r,c}(S) \iff r \in \text{rows}(S) \text{ and } c \in \text{cols}(S).$$

Lemma 2.1 (Support persistence, in equality form). *For every subset P and every letter (f, g) ,*

$$\text{rows}(\Delta_{f,g}(P)) = \text{rows}(P) \cup f(\text{rows}(P)), \quad \text{cols}(\Delta_{f,g}(P)) = \text{cols}(P) \cup g(\text{cols}(P)). \quad (2.4)$$

In particular $\text{rows}(P) \subseteq \text{rows}(\Delta_{f,g}(P))$ and $\text{cols}(P) \subseteq \text{cols}(\Delta_{f,g}(P))$, so every subset reachable from $\{(r, c)\}$ is valid relative to (r, c) .

Proof. The image cell $(i, g(j))$ contributes the first coordinate i , so it contributes exactly $\text{rows}(P)$; the image cell $(f(i), j)$ contributes exactly $f(\text{rows}(P))$. Their union is the first equality, and the second is the same calculation with the coordinates exchanged. Inclusion follows, and applying it successively to the letters of a reaching word, starting from the singleton, gives validity. $\square$

Corollary 2.2 (Supports of a predecessor). *If $\Delta_{f,g}(P) = S$, then $\text{rows}(P) \subseteq \text{rows}(S)$ and $\text{cols}(P) \subseteq \text{cols}(S)$.*

Proof. Immediate from (2.4). $\square$

The equality form of the lemma is worth stating separately, because the sharper word-length bound of Theorem 7.5 is an induction on support counts and needs to know that supports never shrink along a reduction, not merely that they never shrink along a reaching word.

Validity does not require $(r, c) \in S$, and the failure is not a corner case. On a 2×2 grid, the letter whose two components are both the transposition sends $\{(0, 0)\}$ to $\{(1, 0), (0, 1)\}$, which meets row 0 and column 0 without containing $(0, 0)$. Losing sight of this distinction can invalidate an induction that silently changes its initial point.

There is a second, execution-level proof that reachable implies valid, which is worth recording because it exposes where completeness of the operand automata is used. One possible nondeterministic execution gives every input letter to the column operand, leaving the row coordinate equal to r throughout; another gives every letter to the row operand, leaving the column coordinate equal to c . Both executions are available only because the operand transition functions are total.

Proposition 2.3 (Counting valid subsets). *There are exactly $F(m, n)$ valid subsets relative to a specified root. Consequently $\text{sc}_{\text{shuf}}(m, n) \leq F(m, n)$.*

Proof. Rename the root as $(0, 0)$. Counting by inclusion and exclusion, the subsets missing row 0 number $2^{(m-1)n}$, those missing column 0 number $2^{m(n-1)}$, and those missing both number $2^{(m-1)(n-1)}$, so the valid subsets number

$$2^{mn} - 2^{(m-1)n} - 2^{m(n-1)} + 2^{(m-1)(n-1)}. \quad (2.5)$$

Equivalently, and this is the form used later: subsets containing $(0, 0)$ contribute 2^{mn-1} ; a valid subset that omits $(0, 0)$ must contain a nonempty subset of the other $n - 1$ cells of row 0, and a nonempty subset of the other $m - 1$ cells of column 0, while all $(m - 1)(n - 1)$ remaining cells are optional. The two disjoint cases give exactly (1.1), and factoring $2^{(m-1)(n-1)}$ out of (2.5) gives the same expression. Lemma 2.1 bounds the number of reachable determinized states, and minimizing the deterministic automaton cannot increase that number. $\square$

This rederives the upper bound of [1]. The actual work is to show that validity is sufficient for reachability in the claimed range.

3 Distinguishability requires no computation

Reachability is not by itself a state-complexity lower bound: two reachable subset states might accept exactly the same suffixes. This section proves distinguishability four times. Three of the four proofs are over the full transformation-pair alphabet and differ in their machinery; the fourth is a different theorem over a *fixed three-letter* alphabet, and yields a strictly stronger corollary. None of the four uses any computation, and none depends on any later section. Throughout, assume $m, n \geq 2$.

For the three universal-alphabet arguments, choose one final state $a \in R$ and one final state $b \in C$, so that a subset is accepting precisely when it contains (a, b) .

3.1 A forward two-letter construction

Lemma 3.1 (A two-letter cell test). *For every cell $(p, q) \in R \times C$, there is a word of length two accepted from the singleton $\{(p, q)\}$ and from no other singleton.*

Proof. Choose $a_0 \in R \setminus \{a\}$, $b_0 \in C \setminus \{b\}$, and $c_0 \in C \setminus \{q\}$. Let the second letter be (h, k) , where

$$h(i) = a_0 \quad \text{for every } i, \quad k(j) = \begin{cases} b, & j = q, \\ b_0, & j \neq q. \end{cases}$$

Because h never reaches a , a single source cell can produce the final cell (a, b) under this letter only through its second branch. That branch reaches (a, b) exactly from (a, q) .

Let the first letter be (f, g) , where

$$f(i) = \begin{cases} a, & i = p, \\ a_0, & i \neq p, \end{cases} \quad g(j) = c_0 \quad \text{for every } j.$$

The second branch of this letter never reaches column q . Its first branch reaches (a, q) exactly from (p, q) . Therefore the two-letter word $(f, g)(h, k)$ is accepted from precisely the desired source cell. $\square$

Corollary 3.2 (All different subsets are distinguishable). *Every two distinct subsets of $R \times C$ are distinguishable in the universal shuffle automaton with final cell (a, b) .*

Proof. Choose a cell in the symmetric difference of the two subsets. The word from Lemma 3.1 is accepted from a subset exactly when the subset contains that cell, because the transition of a union is the union of the transitions. $\square$

3.2 The same statement by a cellwise existential predecessor

Parallel account #1. The next proof establishes Lemma 3.1 again, but it constructs the two letters *backwards* from the accepting cell, using an explicit preimage computation instead of a forward case analysis. What it buys is a closed formula, $\Delta_A^{-1}[\{(a, b)\}] = \{(p, b)\}$, that identifies the whole intermediate state rather than only its relevant member; that formula is reused in the single-cell separation tests of Section 11. It also isolates exactly where $m, n \geq 2$ is needed — one must be able to choose an image avoiding a prescribed coordinate.

Second proof of Lemma 3.1. For a letter $A = (f_A, g_A)$ write $\Delta_A^{-1}[T]$ for the *cellwise existential predecessor* of T : the set of individual cells having at least one A -successor in T . It is not an inverse function on subset states.

Choose f_A with $f_A^{-1}(\{a\}) = \{p\}$ and choose g_A whose image avoids b ; both are possible because each coordinate set has at least two elements. Then

$$\Delta_A^{-1}[\{(a, b)\}] = (f_A^{-1}(\{a\}) \times \{b\}) \cup (\{a\} \times g_A^{-1}(\{b\})) = \{(p, b)\}.$$

For a second letter $B = (f_B, g_B)$, choose f_B whose image avoids p and choose g_B with $g_B^{-1}(\{b\}) = \{q\}$. Then $\Delta_B^{-1}[\{(p, b)\}] = \{(p, q)\}$. Hence the word BA , read B first, is accepted from exactly the singleton $\{(p, q)\}$; by the union identity

$$\Delta_{f,g}(X \cup Y) = \Delta_{f,g}(X) \cup \Delta_{f,g}(Y), \quad (3.1)$$

which holds for a letter and hence for a word, a subset accepts it exactly when it contains that cell. $\square$

Remark 3.3. The two letters depend on the tested cell. Neither proof says that one fixed pair of letters works for all tests, or that a two-letter alphabet attains the state-complexity bound.

3.3 The same statement by enumerating run allocations

Parallel account #2. The third proof of the two-letter test reasons about the *shuffle semantics* directly instead of about subset transitions: it enumerates the four ways of allocating the two letters between the two operands and eliminates three of them. What it buys is that it never manipulates the determinized transition at all, so it is the argument to check if one suspects an error in (2.2) itself; it also exhibits the separating letters as an explicit table.

Fix the final states $a = m - 1$ and $b = n - 1$, and use state 0 as a nonfinal state in each coordinate; these are distinct because $m, n \geq 2$. Fix a cell (p, q) and choose any $d \in C \setminus \{q\}$. Define two letters by

$$\begin{aligned} \alpha(i) &= \begin{cases} a, & i = p, \\ 0, & i \neq p, \end{cases} & \gamma(j) &= d, & A &= (\alpha, \gamma), \\ \zeta(i) &= 0, & \beta(j) &= \begin{cases} b, & j = q, \\ 0, & j \neq q, \end{cases} & B &= (\zeta, \beta). \end{aligned}$$

Third proof of Lemma 3.1. Consider a singleton input (i, j) and the four ways to allocate the letters A then B to the row or the column operand. If B goes to the row operand, the final row coordinate is 0, so acceptance is impossible; this eliminates two allocations. If both letters go to the column operand, A first puts the column at $d \neq q$ and B then puts it at 0, again rejecting. The only remaining allocation gives A to the row operand and B to the column operand, and its final cell is $(\alpha(i), \beta(j))$, which equals (a, b) exactly when $i = p$ and $j = q$. The statement for arbitrary subsets follows by (3.1). $\square$

3.4 A fixed three-letter alphabet, and a stronger corollary

Parallel account #3. The following is not another proof of Lemma 3.1: it is a different theorem. It uses a *fixed* alphabet of three letters, whatever m and n are, and it produces closed-form distinguishing words. What it buys is twofold. First, it is the only distinguishability result here that survives if one insists on a small alphabet, and it therefore reproduces the construction of [1] inside this manuscript instead of importing it. Second, its conclusion is strictly stronger: *all* 2^{mn} subsets are pairwise distinguishable, reachable or not. Nothing in the other three arguments gives that.

Make $(m - 1, n - 1)$ the unique accepting cell and consider these three letters of the universal alphabet:

Letter	Row transformation	Column transformation
a	$i \mapsto i + 1 \pmod{m}$	$j \mapsto 0$
b	$i \mapsto 0$	$j \mapsto j + 1 \pmod{n}$
c	$0 \mapsto 1, \text{ all } i \geq 1 \mapsto 0$	$j \mapsto n - 1$

Lemma 3.4 (Singleton-distinguishing words). *For every cell (i, j) there is a word $w_{i,j}$ over $\{a, b, c\}$ accepted from exactly that cell and from no other.*

Proof. For a set of target cells, a letter's preimage consists of all sources with at least one transition into it. Three uniqueness observations suffice. First, when $r \geq 1$ and $j \geq 1$, the unique a -predecessor of (r, j) is $(r - 1, j)$, because the column branch of a always lands in column zero. Second, when $r \geq 1$ and $j \geq 1$, the unique b -predecessor of (r, j) is $(r, j - 1)$, because the row branch of b always lands in row zero. Third, the unique c -predecessor of $(1, 0)$ is $(0, 0)$, since $n - 1 \neq 0$ and the only row sent to row one is row zero.

Starting from the unique final cell and working backwards with these observations gives

$$w_{i,j} = \begin{cases} a^{m-1-i}b^{n-1-j}, & j \geq 1, \\ b a^{m-1-i}b^{n-2}, & j = 0, i \geq 1, \\ c b a^{m-2}b^{n-2}, & i = j = 0. \end{cases} \quad (3.2)$$

All exponents are nonnegative. In the first line the backward steps stay in positive columns and the b steps take place in the last row. In the second line the extra initial b moves the unique predecessor from column one to column zero in a positive row. The last line uses the unique c -predecessor of $(1, 0)$. $\square$

Corollary 3.5 (All subsets, reachable or not). *All 2^{mn} subsets of $R \times C$ are pairwise distinguishable, whether or not they are reachable.*

Proof. If two subsets differ, choose a cell in their symmetric difference. The word w of Lemma 3.4 is accepted from a subset exactly when the subset contains that cell. $\square$

3.5 Minimality of the operands

Both component automata over $\Gamma_{m,n}$ are themselves minimal. Every component state is reachable using a constant transformation. Two different states are distinguished by a transformation sending one to its unique final state and the other to a nonfinal state. Thus their state complexities really are m and n , not merely bounded by these numbers.

It follows that proving reachability of all valid subsets in the universal model proves equality in (1.1). The proof optimizes the number of states, not the number of alphabet symbols: a separating *word* of length two does not mean that a two-symbol alphabet suffices for all separations and all reaching words.

4 Root-sensitive reductions

This section develops the reductions used by the induction. Containment and degree-one reductions are part of the prior approach [1, Lemmas 8–9]; complete versions are given here so that root positions and the exact number of seed letters are explicit.

4.1 Relabeling and restriction to a subgrid

Let $\pi : R \rightarrow R'$ and $\sigma : C \rightarrow C'$ be bijections, and write

$$(\pi, \sigma)S = \{(\pi(i), \sigma(j)) : (i, j) \in S\}.$$

Direct substitution into (2.2) gives the covariance identity

$$(\pi, \sigma)\Delta_{f,g}(S) = \Delta_{\pi f \pi^{-1}, \sigma g \sigma^{-1}}((\pi, \sigma)S). \quad (4.1)$$

In particular, renaming coordinates also renames the root. It does *not* generally give a word from the old root to the renamed subset.

Suppose $R_0 \subseteq R$ and $C_0 \subseteq C$ contain the root coordinates, and $S \subseteq R_0 \times C_0$. A word on this smaller grid can be lifted to the large grid by extending each of its functions to fix all excluded coordinates. Starting in the smaller grid, neither transition branch can leave it. Hence a reaching word in the smaller grid is also a reaching word in the full grid.

This removes every empty row or column of a valid target without adding letters. The distinguished row and column are never empty, so their removal is not an issue.

4.2 The one-dimensional base cases

If $m = 1$, a valid subset is $\{0\} \times D$ for a set $D \subseteq C$ that contains c . Starting from $\{(0, c)\}$, add each desired column $j \in D \setminus \{c\}$ by taking $f = \text{id}$ and letting g send c to j while fixing every other column. The identity branch retains every existing cell, and the other branch adds only $(0, j)$.

This uses exactly $|S| - 1$ letters. The case $n = 1$ is symmetric. These are reachability base cases, not an assertion that the state-complexity formula is tight for all one-state operands; Section 13 explains the distinction.

4.3 Deleting a contained copy

Define the row and column neighborhoods of S by

$$N_i(S) = \{j : (i, j) \in S\}, \quad M_j(S) = \{i : (i, j) \in S\}.$$

Suppose no row or column is empty.

Lemma 4.1 (Comparable-row reduction). *If $i \neq k$ and $N_i(S) \subseteq N_k(S)$, then, after possibly reversing i, k in the equality case, there is a valid T with $|T| < |S|$ and a letter (f, id) such that $\Delta_{f, \text{id}}(T) = S$. The same statement holds with rows and columns exchanged.*

Proof. Let (r, c) be the root. If $N_i(S) = N_k(S)$ and $k = r$, interchange i and k . After this choice, either $k \neq r$ or the containment is strict. Set

$$T = S \setminus (\{k\} \times N_i(S)), \quad f(x) = \begin{cases} k, & x = i, \\ x, & x \neq i. \end{cases}$$

The removed set is nonempty. The identity column branch retains T . The row branch copies the retained row i to precisely the removed cells of row k , while producing no cell outside S . Thus the image is exactly S .

If $k \neq r$, the distinguished row is unchanged. If $k = r$, strict containment leaves the nonempty difference $N_k(S) \setminus N_i(S)$ in that row. Finally, deleting a cell (k, c) cannot empty the distinguished column: the retained cell (i, c) accompanies every such deletion. Therefore T remains valid. The column version follows by transposition. $\square$

The equality case is worth retaining in the statement. Blindly deleting the containing copy can erase the distinguished row when the two rows are equal. A choice of direction repairs this without changing the reduction.

Note that the predecessor T of Lemma 4.1 need *not* have full support: it is a proper subset of S and may have lost a row entirely. Root-specific validity is enough here precisely because this reduction is never transported by a relabeling. The contrast with Definition 5.4, where full support is demanded, is the whole reason the two are different objects.

4.4 A degree-one row or column gives an isolated edge

After all comparable neighborhoods have been excluded, any degree-one row or column belongs to an isolated cell. Indeed, if $M_q(S) = \{p\}$ and row p also meets column q' , then $M_q(S) \subseteq M_{q'}(S)$, contrary to incomparability. The transposed argument handles a degree-one row.

Lemma 4.2 (Isolated-cell reduction). *Assume $m, n \geq 2$, all rows and columns of S are nonempty, and (p, q) is isolated in both its row and its column. A reaching word for $S \setminus \{(p, q)\}$ in the complementary $(m-1)$ -by- $(n-1)$ grid can be lifted to a reaching word for S , with at most two additional letters. The smaller root can be chosen so that its target is valid.*

Proof. Let the original root be (r, c) . Choose

$$r' = \begin{cases} r, & p \neq r, \\ \text{any element of } R \setminus \{p\}, & p = r, \end{cases} \quad c' = \begin{cases} c, & q \neq c, \\ \text{any element of } C \setminus \{q\}, & q = c. \end{cases}$$

The target $S_0 = S \setminus \{(p, q)\}$ meets every remaining row and column, so it is valid relative to (r', c') .

Let a be the letter whose row function swaps p, r' and whose column function swaps q, c' , fixing all other coordinates. On their two-by-two rectangle, one application exchanges the two diagonal orientations of a two-cell set. Starting from the original singleton gives the following seed words:

$p = r$	$q = c$	Desired seed	Word
no	no	$\{(p, q), (r', c')\}$	a^2
yes	no	$\{(p, q), (r', c')\}$	a
no	yes	$\{(p, q), (r', c')\}$	a
yes	yes	$\{(p, q), (r', c')\}$	a^2

These four claims also follow by applying (2.1) twice explicitly. In the mixed cases the root is on the opposite diagonal from the desired seed, so one letter, not two, is required.

Take a word reaching S_0 from (r', c') in the smaller grid. Extend each letter to fix p and q . The isolated cell then remains fixed, and every cell of the other component remains within the complementary grid. Since transitions distribute over union by (3.1), appending this extended word to the seed word reaches exactly $S_0 \cup \{(p, q)\} = S$. $\square$

5 Hard matrices and spanning predecessor certificates

5.1 The irreducible targets

Definition 5.1. For $m, n \geq 2$, a subset $S \subseteq [m] \times [n]$ is *hard* if every row and column has at least two cells, distinct row neighborhoods are incomparable under inclusion, and distinct column neighborhoods are incomparable under inclusion. Write $\text{Hard}_{m,n}(S)$ for this property.

The word “hard” is local terminology for the remainder after the elementary reductions, not a computational complexity classification. In particular, a hard matrix is valid relative to *every* possible root.

A valid target that is not one-dimensional either has an empty row or column, has comparable neighborhoods, has an isolated degree-one cell after comparability has been excluded, or is hard. Thus the preceding lemmas exhaust the non-hard cases.

For a hard target the columns are distinct nonempty proper subsets of $[m]$. Nonemptiness and distinctness are immediate. A full column would contain any other column, and $n \geq 2$, so a full column is impossible. Hence, up to column ordering, a hard matrix is a family

$$\mathcal{A} = \{M_0(S), \dots, M_{n-1}(S)\} \subseteq \mathcal{P}([m]) \setminus \{\emptyset, [m]\}$$

that is an antichain and also satisfies the row conditions.

5.2 Why six rows need at most twenty columns

Lemma 5.2 (Antichain width bound). *An antichain $\mathcal{A} \subseteq \mathcal{P}([m])$ has at most $\binom{m}{\lfloor m/2 \rfloor}$ members. In particular, a hard matrix with at most six rows has at most twenty columns.*

Proof. For a fixed $A \subseteq [m]$, the permutations of $[m]$ whose first $|A|$ entries are exactly the elements of A number $|A|!(m - |A|)!$. Prefix sets of one permutation form a chain, so an antichain can occur among these prefix sets at most once. Counting over all $m!$ permutations gives

$$\sum_{A \in \mathcal{A}} |A|!(m - |A|)! \leq m!, \quad \text{hence} \quad \sum_{A \in \mathcal{A}} \binom{m}{|A|}^{-1} \leq 1.$$

Each summand is at least $\binom{m}{\lfloor m/2 \rfloor}^{-1}$. This proves the assertion. For $m \leq 6$ the largest of these middle binomial coefficients is $\binom{6}{3} = 20$. $\square$

This is the usual antichain bound of Sperner [3], with a self-contained counting proof included to identify precisely the finite search space. Applying the same argument to rows also bounds the possible height for a given width. No assertion about reachability is hidden in the antichain theorem.

Remark 5.3 (The same bound, probabilistically). The counting proof above has a one-line probabilistic phrasing: choose a uniformly random permutation of $[m]$; a fixed k -element subset is an initial segment of it with probability $\binom{m}{k}^{-1}$, and at most one member of an antichain can be an initial segment, since initial segments are nested, so $\sum_{A \in \mathcal{A}} \binom{m}{|A|}^{-1} \leq 1$. This is the same mathematics as the proof above, not a second theorem, and is recorded here only because one of the source computations was documented in that language.

In fact the finite search space is already bounded without Sperner's theorem at all. For a hard matrix, every column is a proper subset of $[m]$ of size at least two, and there are only $2^m - m - 2$ such subsets — fifty-six when $m = 6$. That crude count already makes the recursion over column families finite; Lemma 5.2 merely makes it small.

5.3 A root-independent local proof object

Definition 5.4 (Spanning predecessor). A spanning predecessor certificate for $S \subseteq [m] \times [n]$ is a triple (f, g, T) such that

$$f : [m] \rightarrow [m], \quad g : [n] \rightarrow [n], \quad T \subseteq [m] \times [n], \quad (5.1)$$

$$\text{rows}(T) = [m], \quad \text{cols}(T) = [n], \quad (5.2)$$

$$|T| < |S|, \quad (5.3)$$

$$\Delta_{f,g}(T) = S. \quad (5.4)$$

There is no requirement that $T \subseteq S$.

If every smaller valid subset is already reachable, such a certificate proves reachability of S with one additional letter. Full support is stronger than validity, and this extra strength is what makes the finite classification economical.

Lemma 5.5 (Transport of a certificate). *The existence of a spanning predecessor certificate is invariant under arbitrary row and column bijections.*

Proof. Transport T by (π, σ) and transport f, g by conjugation. Cardinality and full support are preserved by bijections; the image identity is preserved by (4.1). $\square$

Notice what the lemma does *not* say: a reaching word from one chosen root is not asserted to remain a reaching word from that same root after arbitrary renaming. Instead, the transported predecessor is valid for the actual root, and the induction supplies its reaching word there. This separates the symmetry argument from the rooted reachability argument.

5.4 An exact inverse formulation

For fixed f, g and target S , define the allowable source cells

$$P_{f,g}(S) = \{(i, j) : (f(i), j) \in S \text{ and } (i, g(j)) \in S\}.$$

Every predecessor must be contained in this set. Make a multigraph with vertex set S , placing one tagged edge for each $(i, j) \in P_{f,g}(S)$ between $(f(i), j)$ and $(i, g(j))$. An edge may be a loop, and different source cells may produce the same pair of endpoints; their source tags are retained.

Proposition 5.6. *A set $T \subseteq P_{f,g}(S)$ satisfies $\Delta_{f,g}(T) = S$ exactly when the edges tagged by T cover all vertices of this multigraph. The predecessor is spanning exactly when these edges include every row tag and every column tag at least once.*

Proof. The endpoints of the edge tagged by (i, j) are precisely the two cells produced by that source. Taking the union of endpoints therefore gives $\Delta_{f,g}(T)$. The final statement is just the definition of support in terms of the tags. $\square$

This interpretation explains why local search is much smaller than a breadth-first search through the entire subset automaton. It also warns against replacing the problem by an unqualified ordinary edge-cover problem: the support requirements concern source tags, not only covered vertices.

The unqualified problem is nevertheless solved exactly, and the solution makes the warning quantitative.

Proposition 5.7 (Minimum unconstrained cover). *Suppose every vertex of the multigraph above is incident with at least one tagged edge, let H_0 be that multigraph with its loops discarded, and let $\nu(H_0)$ be its maximum matching size. Then the minimum number of tagged edges covering all vertices is $|S| - \nu(H_0)$.*

Proof. For the upper bound, start with a maximum matching and cover each unmatched vertex by one incident edge or loop. No two unmatched vertices are joined by a nonloop edge, since that would enlarge the matching; so the number of selected edges is at most $\nu(H_0) + (|S| - 2\nu(H_0)) = |S| - \nu(H_0)$.

For the lower bound, take any cover and delete redundant edges. Every remaining nonloop edge then has an endpoint covered by no other selected edge, so the selected nonloop components are stars and a selected loop is an isolated one-vertex component. Choosing one edge from each of the s star components gives a matching, so $s \leq \nu(H_0)$; and the cover has exactly $|S| - s \geq |S| - \nu(H_0)$ edges. $\square$

Proposition 5.7 is *not* a reachability proof, and the gap is exactly the support constraint: a minimum edge cover may correspond to a predecessor missing a row or a column, which is useless for transport by Lemma 5.5. The certificates below therefore require full support explicitly, and the databases of Section 8 record witnesses, not matching numbers.

5.5 A maximal candidate when the letter is a pair of permutations

If one restricts attention to letters whose two components are permutations, the search collapses to arithmetic on finite sets, with no matching solver and no external solver of any kind. Write $A_j = M_j(S)$ for the target columns. Then (2.2) makes the allowable source set of the previous subsection a column-wise intersection:

$$P_j = f^{-1}(A_j) \cap A_{g(j)}, \quad (5.5)$$

and P , the subset with these columns, is the *largest* possible predecessor for the fixed letter (f, g) . Its image has columns

$$M_j(\Delta_{f,g}(P)) = f(P_j) \cup P_{g^{-1}(j)}. \quad (5.6)$$

A candidate is therefore tested with set intersections, permutation images, and equality of finite sets.

Lemma 5.8 (Equal-cardinality repair). *Let S be hard, let f, g be permutations, let P be given by (5.5), and suppose $\Delta_{f,g}(P) = S$ but $|P| = |S|$. Then some $P' \subsetneq P$ with $|P'| = |S| - 1$ still satisfies $\Delta_{f,g}(P') = S$ and still has full support.*

Proof. Since $P_j \subseteq f^{-1}(A_j)$ for every j , the equality $|P| = |S|$ forces equality in every one of those inclusions; both branches of (2.2) are then bijections from P onto S . Pick an occupied cell of P whose two image cells are distinct and delete it. Each of its two images is still produced, by the other branch applied to a different cell, so the image remains S . Because S is hard, every row and column degree of S is at least two, and P inherits the same row-degree multiset and the same column degrees; deleting a single cell therefore cannot empty a row or a column. $\square$

Thus, when the search is restricted to permutation pairs, a strict predecessor exists whenever the maximal candidate reproduces the target, including in the tight case. This is the entire discovery procedure used for database C in Section 8.4, and it needs no solver. A failure of this restricted strategy on some target would not refute reachability: a nonpermutation letter, or a route through an intermediate subset of equal or larger cardinality, might still work. Compare Section 10.6, where a certificate in database A is exhibited that no permutation-restricted search could find.

6 A fifth reduction: balanced triples

The four reductions so far — empty axes, containment, isolated cell, and the certificate step itself — leave the entire hard class as the finite obligation. One further *analytic* reduction removes a large part of that class before any table is consulted. It is not needed for Theorem 1.1; it gives a second, materially different route to it, with a much smaller finite remainder.

Definition 6.1 (Balanced triple). Three distinct columns of S , with neighborhoods $U, V, W \subseteq [m]$, form a *balanced triple* when

$$U \cup V = V \cup W = W \cup U. \quad (6.1)$$

Equivalently: among these three columns, no row occurs exactly once.

Lemma 6.2 (Balanced-triple contraction). *Let S be hard and contain a balanced triple. Then there is a subset T with full row and column support and $|T| \leq |S| - 3$, and a letter whose row component is the identity and whose column component is a three-cycle, with $\Delta_{\text{id},g}(T) = S$.*

Proof. Let the three columns be a, b, c with neighborhoods U, V, W . Define T by

$$M_a(T) = U \cap V, \quad M_b(T) = V \cap W, \quad M_c(T) = W \cap U,$$

leaving every other column of S unchanged, and let g be the three-cycle $a \mapsto b \mapsto c \mapsto a$ fixing all other columns, with $f = \text{id}$.

The output in column a is

$$(U \cap V) \cup (W \cap U) = U \cap (V \cup W) = U,$$

the last equality because $U \subseteq V \cup W$, which is exactly what (6.1) says. The other two columns are the same calculation cyclically, and all remaining columns are fixed by both branches. Hence $\Delta_{\text{id},g}(T) = S$.

Each intersection is a proper subset of the corresponding original column, because the columns are pairwise incomparable; this gives $|T| \leq |S| - 3$. Each is also nonempty: if $U \cap V = \emptyset$, then (6.1) gives $V \subseteq U \cup W$ and hence $V \subseteq W$, contradicting incomparability. So no column of T disappears. Finally, a row appearing among these three columns appears in at least two of them, and any pair of positions in a three-cycle is one of its consecutive pairs, so that row survives in at least one of the three intersections; rows occurring only in other columns are untouched. Therefore T has full support. $\square$

For example, the three columns $\{0, 1\}, \{1, 2\}, \{0, 2\}$ contract to the three singleton intersections $\{1\}, \{2\}, \{0\}$. The identity is dimension-independent: nothing in the proof mentions six.

Definition 6.3 (Residual core). A *residual core* is a hard matrix containing no balanced triple.

Lemma 6.2 supplies a spanning predecessor certificate, in the sense of Definition 5.4, for every hard matrix that is not residual. Consequently the finite obligation may be narrowed from all hard matrices to residual cores alone.

Remark 6.4 (Hereditary pruning, and what may not be pruned). Balanced-triple-freeness is *hereditary* under deleting a column: once a partial column family contains a balanced triple, adding more columns cannot repair it. This is what allows a column-by-column enumeration of residual cores to prune a prefix as soon as it acquires a balanced triple, and it is what makes the resulting search small. Row degree and row incomparability are emphatically *not* hereditary in this way — adding a column can repair a failure of either — so they are tested at each node and never used to prune a prefix. This distinction is essential to completeness, and it is the single place where a plausible-looking optimization would silently break the enumeration.

7 The finite-to-infinite theorem

The proof architecture can be stated independently of six and independently of the method used to find the certificates.

Definition 7.1. For $h \geq 1$, let $\mathcal{P}_h$ be the assertion that every hard matrix with $2 \leq m \leq h$ rows, of any width, has a spanning predecessor certificate. By Lemma 5.2, this assertion concerns only finitely many matrices up to coordinate permutations.

Theorem 7.2 (Finite certificate principle). *Assume $\mathcal{P}_h$. For every $1 \leq m \leq h$, every $n \geq 1$, every root (r, c) , and every valid $S \subseteq [m] \times [n]$, there is a word over $\Gamma_{m,n}$ reaching S from $\{(r, c)\}$. Moreover, it can be chosen with length at most*

$$B(m, n, S) = |S| + \min(m, n) - 2. \quad (7.1)$$

The same conclusion holds when $\min(m, n) \leq h$, by transposition.

Proof. We prove reachability and the length bound together, for all roots, by strong induction on the lexicographically ordered pair

$$(m + n, |S|) \in \mathbb{N}^2.$$

The induction is well founded. Every dimension-reducing step decreases its first component; every other recursive step decreases its second.

If $m = 1$ or $n = 1$, Section 4 gives a word of length $|S| - 1 = B(m, n, S)$.

Suppose $m, n \geq 2$. If a row or column is empty, delete all empty rows and columns, keeping the actual root, and obtain a smaller grid of dimensions $m' \leq m$, $n' \leq n$, with $m' + n' < m + n$. The cardinality of the target is unchanged. The induction supplies a word, and its lift adds no letters. Since $\min(m', n') \leq \min(m, n)$, its length is at most $B(m, n, S)$.

We may now assume full support. If there are comparable row or column neighborhoods, Lemma 4.1 gives a valid predecessor T in the same grid, with $|T| \leq |S| - 1$. Reach T by induction and append the specified letter. The total length is at most

$$|T| + \min(m, n) - 2 + 1 \leq B(m, n, S).$$

Assume next that all neighborhoods on each side are incomparable. If some row or column has degree one, the corresponding cell is isolated. Lemma 4.2 reduces the problem to a valid target S_0 in an $(m - 1)$ -by- $(n - 1)$ grid, with $|S_0| = |S| - 1$. The induction reaches this smaller target from the chosen smaller root. Adding at most two seed letters gives a word of length at most

$$\begin{aligned} |S_0| + \min(m - 1, n - 1) - 2 + 2 &= (|S| - 1) + (\min(m, n) - 1) \\ &= B(m, n, S). \end{aligned}$$

In the remaining case S is hard. Assertion $\mathcal{P}_h$ gives a spanning predecessor certificate (f, g, T) . Full support of T makes it valid for the actual root. Because $|T| < |S|$, the induction reaches T . Appending (f, g) reaches S , and the same calculation as in the comparable-neighborhood case proves the length bound.

These cases exhaust every valid target. If only the column dimension is at most h , transpose the matrix, interchange the two function components of every letter, and apply the result already proved. This preserves both validity and word length. $\square$

The potential B assigns one unit to every cell and one additional unit to each available dimension on the smaller side, apart from the base normalization. An isolated-cell step loses one cell and one unit of the smaller dimension, paying for its two possible seed letters. This explains the form of the bound, rather than merely obtaining it from a crude bound on the number of subset states.

7.1 The same induction with a weaker finite hypothesis

The proof of Theorem 7.2 consults $\mathcal{P}_h$ only in its last case, and only for the target actually in hand. So the hypothesis may be weakened by anything that discharges part of that case analytically.

Definition 7.3. For $h \geq 1$, let $\mathcal{R}_h$ be the assertion that every *residual* core (Definition 6.3) with $2 \leq m \leq h$ rows, of any width, has a spanning predecessor certificate.

Theorem 7.4 (Finite criterion, residual form). $\mathcal{R}_h$ implies $\mathcal{P}_h$, and hence implies every conclusion of Theorem 7.2.

Proof. Let S be hard with at most h rows. If S contains a balanced triple, Lemma 6.2 supplies a spanning predecessor certificate directly. Otherwise S is residual and $\mathcal{R}_h$ supplies one. $\square$

This is a strictly weaker finite premise: Section 8.4 discharges $\mathcal{R}_6$ with 453 certificates over widths at most eleven, where $\mathcal{P}_6$ needs more than ten thousand over widths up to twenty.

7.2 A sharper bound in terms of the support

Parallel account #4. The bound $B(m, n, S) = |S| + \min(m, n) - 2$ charges the *grid* dimensions. The next theorem charges only the dimensions actually occupied. What it buys is strictly more: on a target with empty axes — exactly the targets for which the first reduction of the induction fires — it is smaller, and it has the dimension-free corollary $|w| \leq 2E - 2$, which the grid form does not imply. It is proved by exhibiting an explicit potential and accounting for it case by case, so it also makes the shape of the bound visible.

Theorem 7.5 (Support form of the word bound). Assume $\mathcal{P}_h$ (or, by Theorem 7.4, $\mathcal{R}_h$). Let $1 \leq m \leq h$, let S be valid relative to (r, c) , put $E = |S|$, $r_S = |\text{rows}(S)|$, and $c_S = |\text{cols}(S)|$. Then S has a reaching word of length at most

$$\Phi(S) = E + \min(r_S, c_S) - 2. \quad (7.2)$$

In particular $|w| \leq 2E - 2$, since $r_S, c_S \leq E$; and for six rows with $n \geq 6$, $|w| \leq 6n + 4$. The same holds with rows and columns exchanged.

Proof. Run the induction of Theorem 7.2 and account for Φ at each step. On a valid singleton $\Phi = 0$ and the empty word suffices. Deleting empty axes changes neither E nor the two support counts, and adds no letters, so Φ is unchanged. A containment predecessor is a proper subset of S , so by Corollary 2.2 its support counts do not increase while E drops by at least one; hence $\Phi(T) \leq \Phi(S) - 1$, and one letter is appended. A certificate predecessor has full support, so its support counts equal those of S (which is hard and hence has full support), and its cardinality is strictly smaller; again $\Phi(T) \leq \Phi(S) - 1$ for one appended letter. An isolated-cell step deletes exactly one cell, one nonempty row, and one nonempty column, so

$$\Phi(S') = (E - 1) + \min(r_S - 1, c_S - 1) - 2 = \Phi(S) - 2,$$

which pays for its at most two seed letters, while the lift adds none. The one-dimensional base uses $E - 1 = \Phi(S)$ letters, since one of the support counts is 1. Induction on the same rank $(m + n, |S|)$ gives the bound. $\square$

7.3 A dual bound carried through the induction

Parallel account #5. The third form carries *two* estimates through the induction at once and takes their minimum. What the first branch, $2(|S| - 1)$, buys is that it does not mention the dimensions at all, so it is the sharper of the two exactly when the grid is wide relative to the target; neither branch implies the other, and the minimum is smaller than either form above on some targets. The accompanying constructor checks both estimates on every word it emits.

Theorem 7.6 (Dual word bound). *Under the hypothesis of Theorem 7.2, every valid S has a reaching word with*

$$|w| \leq \min\{2(|S| - 1), |S| + \min(m, n) - 2\}. \quad (7.3)$$

Proof. Write $k = |S|$ and $d = \min(m, n)$, and carry both estimates simultaneously through the cases of Theorem 7.2. The singleton uses zero letters, and the one-dimensional construction uses $k - 1$ letters, which satisfies both. Deleting empty axes changes no cell and adds no letter, while d cannot increase. Each containment step, balanced-triple step, and certificate step decreases k by at least one and appends one letter, so both $2(k - 1)$ and $k + d - 2$ survive. The isolated-cell step replaces (k, d) by $(k - 1, d - 1)$ and adds at most two letters, and its two estimates are

$$2((k - 1) - 1) + 2 = 2(k - 1), \quad (k - 1) + (d - 1) - 2 + 2 = k + d - 2.$$

Both bounds therefore hold at every step, hence so does their minimum. $\square$

7.4 Why this is not an extrapolation from finite experiments

There is no untested assumption of the form “the pattern probably continues for larger n .” Every possible target is classified. A target with empty axes, comparable supports, or a degree-one axis is reduced analytically; a target containing a balanced triple is reduced analytically; and every target not reduced in these ways is one of the finitely many encoded shapes, modulo row and column relabeling. The finite coverage results then handle all of those, and the induction applies for any finite number of columns.

Likewise, a certificate table contains predecessors, not an assertion that those predecessors are already reachable. Their reachability follows from the strictly decreasing induction. That is why a positive table of local equations is sufficient to prove a global statement about all reaching words. The width bounds — twenty for six rows, or eleven after balanced-triple removal — are bounds on the width of an irreducible *obstruction*, not on the widths the theorem covers. Section 10 exhibits a 6×100 target handled with no hundred-column certificate.

8 The finite certificate computations

The induction of Section 7 needs one finite input: $\mathcal{P}_6$, or the weaker $\mathcal{R}_6$. This section supplies three separate discharges of that input, from three certificate databases:

	Classified objects	Records	Widths	Discovery
A	hard matrices, orbit-closure coverage	10,642	≤ 20	SMT
B	hard matrices, degree-sorted canonical	10,619	≤ 20	SMT
C	residual cores (no balanced triple)	453	≤ 11	solver-free

A and B classify the same objects by two different symmetry-reduction correctness proofs; C classifies a strictly smaller class, made available by Lemma 6.2. Any one of the three suffices for Theorem 1.1. All three share the encoding of the next subsection.

Parallel account #0. What each database buys. A is the uncompressed baseline: it certifies the whole hard class directly, and it is the only one containing certificates with non-injective row maps, which no permutation-restricted search could find. B is an independent re-derivation of the same finite lemma over a provably minimal representative set, sharing no witness with A, and it additionally verifies that its row maps are permutations. C is the compressed route: a fifth analytic reduction makes the finite obligation small enough for a deterministic, solver-free search, so that one path through the entire result has no external solver anywhere in it. Section 8.5 records precisely why none reduces to another.

8.1 Encoding the targets

A column $A \subseteq [m]$ is encoded by

$$b(A) = \sum_{i \in A} 2^i.$$

An unordered family of distinct columns is encoded by

$$X(\mathcal{A}) = \sum_{A \in \mathcal{A}} 2^{b(A)}. \quad (8.1)$$

For $m \leq 6$, every nonempty proper column has a code between 1 and 62. Thus the entire family fits in an unsigned 64-bit integer. There is no shift by 64, and the prohibited full column at code 63 is not needed. The family determines a matrix by arranging its column codes in increasing order. Row permutations act on the binary digits of every column code.

8.2 Database A: orbit-closure coverage

8.2.1 The representative list, and why it need not be minimal

The A search generator grows antichains one column at a time and canonicalizes under row permutations. It first sorts the multiset of row degrees; among row permutations realizing that degree order, it chooses the smallest family integer. For completeness of this *search* enumeration, remove one column from any target antichain: an orbit representative of the smaller antichain occurs at the preceding level, the same relabeling transports the removed column to an allowable extension, and canonicalizing that extension recovers the target orbit. This proves level-by-level coverage of the generator.

The final proof, however, does not rely on this reasoning or on this implementation at all. Coverage is audited by an independent checker that expands the *orbit closure* of each verified representative and compares against a directly enumerated labeled traversal (Section 8.2.5). Because it compares orbit closures and the size of the closed set, rather than a list of minimal representatives, that design is insensitive to redundancy in the representative list: a representative whose orbit is already covered costs a wasted search but cannot create a gap, and no minimality of the list is required, or claimed, anywhere in the argument. This tolerance is a real architectural difference

from database B, whose correctness argument instead *proves* that its representative list is exactly minimal and whose checker consequently rejects a duplicate key outright.

A visible consequence of the same difference is that this classification never transposes. It certifies hard matrices in the orientation in which they arise, including those with fewer columns than rows, where database B orients every target with $m \leq n$ at the outset and recovers the rest by transposition. That accounts exactly for the 23 records by which the two representative sets differ; Section 8.5 identifies them.

8.2.2 The solver query

For each hard representative, the search introduces integers $f_0, \dots, f_{m-1}$ and $g_0, \dots, g_{n-1}$ with the appropriate finite ranges, and Boolean variables t_{ij} indicating whether $(i, j) \in T$. It imposes the following conditions:

$$t_{ij} \implies ((f_i, j) \in S \wedge (i, g_j) \in S), \quad (8.2)$$

$$(r, c) \in S \implies \left(\bigvee_{i=0}^{m-1} (t_{ic} \wedge f_i = r) \right) \vee \left(\bigvee_{j=0}^{n-1} (t_{rj} \wedge g_j = c) \right), \quad (8.3)$$

$$\bigvee_j t_{ij} \text{ for every } i, \quad \bigvee_i t_{ij} \text{ for every } j, \quad (8.4)$$

$$\sum_{i,j} [t_{ij}] < |S|. \quad (8.5)$$

Membership tests involving f_i or g_j are expanded into disjunctions of equalities to the constant coordinates of target cells. Here $[t]$ is 1 when t is true and 0 otherwise.

Equation (8.2) forbids extraneous image cells, while (8.3) requires every target cell to be produced. The last two lines enforce spanning and strict size reduction. Thus a satisfying assignment is exactly a spanning predecessor certificate. No optimization of $|T|$ is required. In the largest shape there are 120 membership Booleans and 26 function-value integers.

The implementation uses Z3 through its documented C API [4], called from Python’s standard-library `ctypes` module. The solver is used only to discover witnesses. Its search heuristics, internal reasoning, and status string are not accepted as the verification of a certificate. An unsuccessful or timed-out search would also not refute the shuffle conjecture.

8.2.3 What was found

The search returned a spanning predecessor for every one of the 10,642 hard representatives generated for $2 \leq m \leq 6$. Of these, 10,573 have six rows. Table 1 summarizes the independent coverage audit. A “labeled family” here retains row labels but forgets column ordering; it is not a count of all labeled rectangular matrices.

All target sizes in the certificate file lie between 6 and 60; predecessor sizes lie between 5 and 38. Every certificate strictly reduces size, and the observed reductions range from 1 to 24 cells. These are properties of the concrete file, not unproved universal size estimates for arbitrary numbers of rows.

The cases below six rows are included rather than treated as black-box computational results from the literature. This makes the inductive certificate argument self-contained once the local and

Table 1: Exhaustive coverage by row count. The antichain column excludes the special singleton families $\{\emptyset\}$ and $\{[m]\}$, neither of which can be hard. The empty family is included.

Rows	Antichains visited	Hard labeled families	Certificates
2	4	0	0
3	18	1	1
4	166	15	5
5	7,579	1,957	63
6	7,828,352	5,237,599	10,573

Table 2: The six-row hard families, classified by number of columns. The first numeric column counts row-permutation representatives.

Columns	Certificates	Hard labeled families
4	1	30
5	21	5,100
6	296	117,821
7	1,141	564,330
8	2,133	1,113,390
9	2,416	1,264,720
10	1,973	998,987
11	1,274	618,712
12	706	323,900
13	349	147,020
14	159	57,645
15	65	19,238
16	25	5,325
17	9	1,170
18	3	190
19	1	20
20	1	1
Total	10,573	5,237,599

coverage audits are accepted.

The full distribution of six-row hard families by number of columns, for every n from 0 through 20, is given in Appendix C; that table also gives the total labeled antichain count at each width, which is the denominator the coverage traversal actually visits. The corresponding five-row histogram is

$$\{4:30, 5:495, 6:850, 7:445, 8:115, 9:20, 10:2\},$$

summing to the 1,957 of Table 1; the four-row histogram is $\{4:8, 5:6, 6:1\}$. All three histograms are recorded in `audit/coverage.json`.

8.2.4 A solver-independent local checker

The Python checker reads each JSON record and validates both its structure and its mathematics. It checks the dimensions, function ranges, distinct column codes, antichain conditions, row and column degrees, and the family encoding. It rejects repeated cells in T . It then computes the two images in (2.2) explicitly and checks

$$\text{rows}(T) = [m], \quad \text{cols}(T) = [n], \quad |T| < |S|, \quad \Delta_{f,g}(T) = S.$$

All these checks use exact integers and finite sets. Explicit exceptions, not disableable Python assertions, enforce the independent checker's conditions. The solver module is not imported by this checker.

After validating a record, the checker exports its target identifier to `audit/verified_targets.tsv`. This ties the subsequent coverage run to targets whose actual certificates were checked. A list of target names without validated witnesses would not suffice.

8.2.5 An independent exhaustive coverage checker

The C++ coverage checker does not use the search generator's canonicalizer, degree ordering, or level-by-level orbit construction. It proceeds in two separate stages.

First, it expands every verified representative under *all* $m!$ row permutations, sorts the resulting family integers, and removes duplicates. Call the resulting orbit-closed set O_m .

Second, it enumerates every antichain of the nonempty proper subsets of $[m]$ directly, with row labels fixed. Candidate column codes are kept in increasing order. At a recursion node, each candidate is considered as the next column; all candidates comparable to it are deleted before descent. Every current family is inspected, not just maximal antichains. The pseudocode is:

```
def visit(family, candidates):
    inspect(family)
    while candidates:
        s = smallest(candidates)
        candidates.remove(s)
        visit(family | {s},
              {t for t in candidates if incomparable(s, t)})
```

Lemma 8.1 (Coverage enumeration is exhaustive). *Starting with the empty family and all nonempty proper column codes, this recursion visits each antichain exactly once.*

Proof. Every recursively added column is larger than the preceding choices and incomparable with them, so every visited family is an antichain. Conversely, list the members of an arbitrary antichain in increasing order. At each step of the recursion, its next member remains a candidate: it is larger than the members already chosen and is incomparable with all of them. Following these choices visits the given family. Strictly increasing order makes this sequence of choices unique. $\square$

Remark 8.2. Two of the source packages phrase this completeness lemma differently — one in terms of a set of “available” masks from which all comparable masks are removed on each descent, the other in terms of a candidate set intersected with the incomparable masks — but the two phrasings describe the same recursion and the same proof. Only this one is given. The same recursion, with one extra pruning rule, is used for database C in Section 8.4.

For every visited hard family, the checker demands membership in O_m . It also compares the number of hard visited families to $|O_m|$. Thus a missing orbit is not concealed by the fact that a canonicalization algorithm never generated it. In the six-row run, all 5,237,599 hard labeled families were in the orbit-closed checked set. The output is recorded in `audit/coverage.json`.

Lemma 8.3 (Finite certificate lemma from A). *Assertion $\mathcal{P}_6$ holds.*

Computer-assisted proof. The independent local checker validated all 10,642 spanning predecessor certificates in `data/certificates.jsonl`. The independent exhaustive coverage checker, whose enumeration is justified by Lemma 8.1, verified that every hard column family for each $m \in \{2, 3, 4, 5, 6\}$ is a row permutation of a checked representative. Column ordering is immaterial by Lemma 5.5. Every hard matrix therefore inherits a spanning predecessor certificate. The complete run reports and executable source are included in the archive. $\square$

8.2.6 Why the two stages are separated, and a one-binary alternative

The two-stage design is deliberate. The first stage alone is not a coverage proof: it validates whatever witnesses it is handed and says nothing about what is missing. The second stage alone is not a witness proof: it audits a list of target identifiers, and a list of names without validated witnesses establishes nothing. The coupling between the stages is the exported file `audit/verified_targets.tsv`, which contains exactly the identifiers of records that survived stage one, and which stage two consumes as its only input. Both stages must exit successfully, and the article’s finite-to-infinite argument is a third, separate obligation.

Parallel account #6. A second checker architecture for the same database is also shipped: `src/verify_all.cpp`, a single self-contained C++17 binary that performs the certificate checks, the orbit expansion, and the labeled-antichain coverage traversal in one process, taking the plain-integer serialization `data/certificates.txt` as its only argument. What it buys is that there is no intermediate file to get out of step, and no way to run one half of the check and report the other: the proof of the finite lemma is one command with one expected line of output. What the two-stage design buys in exchange is that each stage can be reimplemented, rerun, and audited on its own, and that the interface between them is an inspectable artifact. Both are kept.

The expected final line of the single binary is

PASS: 7836132 labeled antichains enumerated; 5239572 hard antichains covered.

These are totals across $m = 1, \dots, 6$ inclusive, so they are larger than the $m \geq 2$ totals of Table 1; the one-row case contributes three antichains and no hard family. Table 3 gives the breakdown.

Table 3: The single-binary run, including the one-row case. “Stored” counts positive records; no minimality of the representative list is required, or claimed, for this proof.

Rows	Stored	All labeled antichains	Hard antichains	Covered
1	0	3	0	0
2	0	6	0	0
3	1	20	1	1
4	5	168	15	15
5	63	7,581	1,957	1,957
6	10,573	7,828,354	5,237,599	5,237,599
Total	10,642	7,836,132	5,239,572	5,239,572

The antichain totals here exceed those of Table 1 by exactly two per row count, because this traversal also visits the two degenerate singleton families $\{\emptyset\}$ and $\{[m]\}$, neither of which can be hard. The two tables are two accountings of one traversal, not two computations.

8.2.7 Two serializations, and a test that exists only because of that

Database A is shipped in three byte-different encodings of the same 10,642 witnesses: `data/certificates.jsonl`, whose predecessor field is an explicit list of coordinate pairs; `data/certificates_masks.jsonl`, whose predecessor field is a list of column bit masks; and `data/certificates.txt`, a header-commented plain-integer file with one whitespace-separated record per line, which is what the single binary reads. Appendix A gives all three layouts.

Parallel account #7. Shipping more than one serialization creates a check that is otherwise impossible: `src/test_artifacts.py` compares the plain-integer file and the mask JSON *record by record*, requiring $(m, n, \text{key}, f, g, \text{predecessor})$ to agree on every line. What it buys is a cross-check of the writers, not of the mathematics: a transcription or column-ordering fault that both checkers would accept, because both would read the same corrupted file, cannot survive agreement between two files written by different code paths. During the preparation of this merged archive the same comparison was also run between the coordinate-pair file and the mask file, converting each record’s coordinate list to column masks; all 10,642 records agreed.

8.3 Database B: a minimal canonical representative set

Database B discharges $\mathcal{P}_6$ a second time, over a different representative set produced by a different symmetry-reduction argument, with a different search run, a different file format, and different checkers. It is not a re-verification of A; the two share no witness.

8.3.1 The degree-sorted canonical convention

Retain a labeled family only when its row-degree sequence is nondecreasing; among the row permutations preserving that degree sequence, select the one minimizing the family integer (8.1).

Lemma 8.4 (Canonical representatives). *Every row-permutation orbit of hard matrices contributes exactly one representative under this convention.*

Proof. Any row-degree sequence can be sorted by a row permutation, so every orbit has at least one retained labeling. All retained labelings in one orbit have the same sorted degree sequence, so a permutation between two of them sends each row to a position of equal degree; that is, it permutes only equal-degree blocks. Minimizing the family integer over exactly those permutations therefore yields the same integer for every retained labeling in the orbit. Distinct orbits cannot produce the same encoded family. $\square$

Parallel account #8. Lemma 8.4 and the orbit-closure argument of Section 8.2.5 are two different theorems about two different objects. Lemma 8.4 says the representative list is *exactly minimal*, one per orbit, which permits a checker that compares representative sets directly and rejects duplicates outright. The orbit-closure argument says nothing about minimality and instead compares *closures*, which permits a redundant list. Neither implies the other, and each is the correctness proof of a different shipped program: a checker built on one assumption would be unsound applied to the other’s data file. Both are kept.

The enumeration itself is the recursion of Lemma 8.1, run over all 2^m column masks, with the hard filters applied at each node: $n \geq m$, no column of size zero or one, every row degree at least two, and row incomparability checked in both directions. Column incomparability is already guaranteed by the recursion.

8.3.2 Enumeration results

Table 4 gives the totals; the file `data/core\_counts.csv` gives a four-way breakdown per (m, n) — all antichains, labeled hard matrices, degree-sorted survivors, and orbits — which no other count in this manuscript supplies.

Table 4: Database B enumeration, oriented so that $m \leq n$. The “all antichains” column includes the two degenerate singleton families, so it exceeds the corresponding column of Table 1 by two.

Rows m	All antichains	Width bound	Core orbits
2	6	2	0
3	20	3	1
4	168	6	5
5	7,581	10	62
6	7,828,354	20	10,551
Total certificate targets			10,619

The 10,551 six-row orbits are distributed by width as

n	Orbits	n	Orbits	n	Orbits
6	296	11	1,274	16	25
7	1,141	12	706	17	9
8	2,133	13	349	18	3
9	2,416	14	159	19	1
10	1,973	15	65	20	1

so widths 7 through 20 account for 10,255 genuinely new representative targets; adding the 296 six-by-six targets and 68 targets from smaller dimensions gives 10,619.

8.3.3 A verifier that regenerates rather than trusts

The C++20 program `src/verify.cpp` both checks every triple and *reruns the entire canonical enumeration*, then compares the full set of target keys — not the number of records — with the set of keys appearing in the certificate file. It rejects trailing tokens, duplicate keys, incorrect column counts, empty source columns, missing source rows, nondecreasing cardinalities, incorrect images, and a row map that is not a permutation. Two checkers sharing one target list would not independently establish coverage; that is exactly why this one regenerates the list.

A second, independently written Python checker, `tests/verify\_certificates.py`, recomputes all 10,619 transition images using sets of ordered pairs rather than output bit masks, and checks the hard conditions directly. For $m \leq 4$ it additionally enumerates *all families of subsets*, not the antichain recursion, and obtains the same canonical sets.

Proposition 8.5 (Finite certificate lemma from B). *Assertion $\mathcal{P}_6$ holds. Moreover, the row transformation in every one of the 10,619 supplied certificates is a permutation, and the decreases $|S| - |T|$ range from 1 through 22.*

Computer-assisted proof. By Lemma 5.2 only finitely many widths arise. The C++ enumerator implements the recursion of Lemma 8.1, the hard filters, and the canonicalization of Lemma 8.4, producing 10,619 representatives. The verifier checks each supplied triple by the explicit image computation and then compares key sets: they are equal, with no missing, duplicate, or extra target. Every triple has full support, strictly smaller cardinality, the exact required image, and a permutation row map. The Python checker reproduces all image computations by a different representation. Lemma 5.5 extends each representative certificate to all labelings. $\square$

Parallel account #9. The additional clause of Proposition 8.5 — that f is always a permutation — is a genuine extra verified property, and it is *false* of database A, where 8,117 of the 10,642 row maps are not injective. It matters because a permutation-only search is far cheaper and needs no solver, and because it is the six-row instance of the stronger structural statement discussed in Section 13. The column map is not always a permutation in B either: 10,113 of its 10,619 column maps fail injectivity.

The role of the two checkers must not be conflated: the Python check of six-row coverage consults the representative manifest, whereas the *exhaustive* six-row coverage computation is the C++ one.

8.4 Database C: residual cores and no solver at all

Database C discharges the weaker premise $\mathcal{R}_6$ of Theorem 7.4. Because Lemma 6.2 has already removed every hard matrix containing a balanced triple, the class that remains is far smaller, and it is small enough that the discovery procedure of Section 5.5 — pure permutation enumeration — is sufficient.

8.4.1 The enumeration

For each $m = 2, \dots, 6$, start from the increasing list of masks of proper subsets of $[m]$ of size at least two. Maintain a chosen family and the remaining larger candidates. On choosing a new column, reject it if it forms a balanced triple with two already-chosen columns; otherwise recurse, retaining only later candidates incomparable with it. At every node with at least m columns, test row incomparability and row degree, and if they hold record a residual core.

Completeness is Remark 6.4 plus the uniqueness argument of Lemma 8.1: every residual family has a unique increasing mask sequence, each of whose prefixes is itself incomparable and balanced-triple-free, so no prefix can be rejected by either pruning rule and the family is reached. Row degree and row incomparability are tested at nodes and never used to prune.

Predecessor columns are stored in the representative’s *original* column order; sorting them independently would generally destroy the certificate. The records also carry a redundant family key, checked for consistency but not assumed by the proof.

Table 5: Exact residual-core enumeration, oriented so that $m \leq n$. Rows are labeled and the columns form an unordered family; “representatives” additionally quotients by row permutations. Every case with $2 \leq m \leq n$, $m \leq 6$, not listed here has no residual core at all.

Rows m	Columns n	Row-labeled column families	Representatives
4	4	7	2
5	5	239	7
5	6	115	4
5	7	10	1
6	6	54,358	136
6	7	83,730	194
6	8	26,565	84
6	9	3,355	19
6	10	284	5
6	11	10	1
Total		168,673	453

The recursion visits 1, 7, 79, 2,715, 584,473 prefix nodes for $m = 2, \dots, 6$. There are 439 six-row representatives, of which 303 have at least seven columns; the remaining 14 representatives have fewer than six rows. The generator tested 690,293 permutation pairs for the six-row representatives before finding all their certificates.

The enumeration does not stop at eleven columns by assumption: it explores every admissible family in the whole pool, and the *absence* of residual cores wider than eleven is an output of the computation, not an input to it.

8.4.2 A checker that needs no compiler, and a generator that needs no solver

Parallel account #10. `residual/verify.py` is a single pure-Python standard-library program that checks the certificates, expands the row-permutation orbits, checks that representative orbits are pairwise *disjoint*, and runs the exhaustive residual enumeration, together with auxiliary checks of the local formulas. What it buys is that the entire finite lemma can be checked with no C++ compiler, no build step, and no third-party package — the weakest prerequisite of any verification path here. Its `-quick` flag checks certificate soundness only and deliberately omits exhaustive coverage; its own output warns that `-quick` must not be represented as verification of the finite lemma.

Parallel account #11. `residual/generate.cpp` is a deterministic C++17 regenerator using integer bit builtins and *no solver of any kind*, implementing the maximal-candidate formula (5.5) with the repair of Lemma 5.8. Its regenerated output was compared byte for byte with the shipped certificate file and found identical. What it buys is that this one route through the whole result — discovery and checking — has zero dependency on Z3 or on any external solver, so the archive is fully regenerable as well as fully checkable. The SMT route of Section 8.2 cannot be replaced by it: the shared database contains non-injective row maps, which a permutation-restricted search cannot produce.

Lemma 8.6 (Finite certificate lemma from C). *Assertion $\mathcal{R}_6$ holds. Moreover, for every residual core with $2 \leq m \leq 6$ the certificate may be taken with both f and g permutations.*

Computer-assisted proof. The file `residual/certificates.tsv` contains 453 records, each of which passes the literal checks of Definition 5.4 together with bijectivity of f and of g . The exhaustive recursion described above covers every residual core up to column ordering, and every such family lies in exactly one verified row-permutation orbit; orbit disjointness is checked, and the number of enumerated families equals the size of the orbit table. Equation (4.1) transports a certificate to any labeling, and full support and cardinality are invariant under relabeling. $\square$

8.5 Why none of the three databases reduces to another

The following comparisons were recomputed directly from the three shipped files during the preparation of this merged archive.

- (a) *Keys.* B’s 10,619 target keys form a strict subset of A’s 10,642. The 23 extra records of A are exactly those whose shape has more rows than columns: twenty-one of shape 6×5 , one of shape 5×4 , and one of shape 6×4 . A certifies each hard matrix in the orientation in which it arises, whereas B orients every target with $m \leq n$ and recovers the others by transposition. Per row count, A stores 1, 5, 63 and 10,573 records at three, four, five and six rows against B’s 1, 5, 62 and 10,551; the differences 1 and 22 are the 5×4 record and the 6×5 and 6×4 records respectively. Within each database every stored record lies in a distinct row-permutation orbit of its shape, as recomputed for this merge. The numbers 10,642 and 10,619 must not be “reconciled” into one: their difference is a real artifact of two different canonicalization conventions, one of which additionally proves minimality of its list while the other proves only that its orbit closures cover.

- (b) *Witnesses.* Of the 10,619 keys present in both databases, *zero* carry the same triple (f, g, T) . Each database is therefore a genuinely independent re-derivation of the other's finite lemma, not a copy with a different header.
- (c) *Structure.* In A, 8,117 of the 10,642 row maps are not injective; in B, all 10,619 are permutations while 10,113 column maps are not; in C, all 453 records have both maps bijective.
- (d) *Class.* C classifies a different class of objects. Its 453 records share only two target keys with A, and the same two with B.
- (e) *Remainder.* The two finite classifications of the irreducible remainder are different objects with different sizes: 5,237,599 six-row hard labeled families up to width twenty, against 168,673 row-labeled residual families up to width eleven. Collapsing them would lose either the balanced-triple compression or the uncompressed baseline that two independent databases certify.

Consequently the finite input of Section 7 is available three times over, and each way is a complete proof by itself: Lemma 8.3, Proposition 8.5, and Lemma 8.6 with Theorem 7.4. The word constructors and supplementary tests of Section 11 provide further checks and usable witnesses, but are not substituted for any of these coverage arguments.

9 Completion of the state-complexity proof

Proof of Theorem 1.1. Lemma 8.3 and Theorem 7.2 show that every valid subset is reachable when $\min(m, n) \leq 6$. Proposition 8.5 gives the same conclusion through an independent finite computation, and Lemma 8.6 with Theorem 7.4 gives it a third time through a smaller one; any one of the three completes this step. Lemma 2.1 shows that no invalid subset is reachable. Hence the universal shuffle automaton has exactly $F(m, n)$ reachable subset states.

For $m, n \geq 2$, give the component automata one final state each. Section 3 proves that the component automata are minimal with respectively m, n states and that all distinct subset states are distinguishable — by any of its four arguments, of which Corollary 3.5 needs only a fixed three-letter alphabet. The resulting shuffle language therefore has state complexity $F(m, n)$. Proposition 2.3 gives the matching upper bound for all operand pairs. This proves the equality.

Finally,

$$\begin{aligned} F(6, n) &= 2^{6n-1} + 31 \cdot 2^{5n-5}(2^{n-1} - 1) \\ &= 63 \cdot 2^{6n-6} - 31 \cdot 2^{5n-5}, \end{aligned}$$

which proves (1.2). □

Corollary 9.1 (Short reaching words). *If $m, n \geq 1$ and $\min(m, n) \leq 6$, every valid subset S of the universal shuffle state space is reachable by a word of length at most*

$$\min\{2(|S| - 1), |S| + \min(m, n) - 2\},$$

and also by one of length at most $|S| + \min(r_S, c_S) - 2$, where r_S and c_S are the numbers of nonempty rows and columns of S . For a six-by- n grid with $n \geq 6$, all three specialize to at most $|S| + 4 \leq 6n + 4$.

Proof. Apply Theorems 7.2, 7.5 and 7.6 with $h = 6$. □

Equivalently, the maximum distance from the initial subset to any reachable subset is at most $mn + \min(m, n) - 2$. This is a statement about the universal alphabet, whose letters already specify complete transformations; it does not transfer to a fixed generating alphabet, because replacing a transformation pair by a word in generators need not preserve the subset action.

For scale, the theorem gives

$$F(6, 6) = 66,605,547,520, \quad F(6, 7) = 4,296,041,037,824,$$

and already at $n = 10$,

$$F(6, 10) = 1,133,816,390,562,611,200.$$

The proof did not enumerate any of these reachable-state sets. It classified the much smaller collection of irreducible targets and used induction to cover the rest.

10 Certificates that can be checked by hand

Four worked six-by-seven examples follow, on *three* different targets, plus wider examples. They are not illustrations of one phenomenon: the first two are two incompatible certificates for the same target, drawn from the two independent databases, and are the cleanest single demonstration that those databases are independent.

10.1 A six-by-seven target, certified from database A

Consider the hard target whose ordered column codes are

$$(12, 17, 18, 24, 33, 34, 36).$$

Its family integer is 94,506,455,040. With rows and columns numbered from zero, its incidence matrix and a predecessor are

$$S = \begin{pmatrix} 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}, \quad T = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}.$$

There are 14 cells in S and 10 in T . The row and column supports of T are full. Define the image lists

$$f = (0, 3, 4, 0, 2, 5), \quad g = (2, 0, 6, 5, 4, 6, 3).$$

Here the i th entry of the first list is $f(i)$, and similarly for g . The two images of each source cell are displayed in Table 6.

Table 6: Direct verification of the six-by-seven predecessor. The union of the last two columns is exactly the target S . Repeated outputs are harmless.

Source (i, j)	$(f(i), j)$	$(i, g(j))$
(0, 4)	(0, 4)	(0, 4)
(1, 0)	(3, 0)	(1, 2)
(1, 3)	(3, 3)	(1, 5)
(2, 1)	(4, 1)	(2, 0)
(2, 2)	(4, 2)	(2, 6)
(3, 1)	(0, 1)	(3, 0)
(4, 0)	(2, 0)	(4, 2)
(4, 6)	(2, 6)	(4, 3)
(5, 4)	(5, 4)	(5, 4)
(5, 5)	(5, 5)	(5, 6)

Every output lies in S , and every 1 of S occurs among the outputs. This checks the image identity without trusting a solver or software. The predecessor is intentionally not constrained to lie inside its target: for instance, $(1, 0) \in T$ but $(1, 0) \notin S$.

10.2 The same target, certified from database B

Database B contains a certificate for this same target — family integer 94,506,455,040 — and it is a different one. Its row map is a permutation:

$$f = (0, 3, 5, 1, 4, 2), \quad g = (4, 2, 2, 2, 1, 0, 6), \quad t = (32, 16, 16, 18, 1, 12, 36),$$

where t lists the predecessor's column codes. As an incidence matrix the predecessor is

$$T' = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix},$$

again with ten cells against the target's fourteen, and again with full row and column support. Separating the two branches of (2.2) by column gives

Column j	0	1	2	3	4	5	6
Vertical branch $(f, \text{id})(T')$	4	16	16	24	1	34	36
Horizontal branch $(\text{id}, g)(T')$	12	1	18	0	32	0	36
Bitwise union	12	17	18	24	33	34	36

so the image is exactly S . In the .cert serialization of Appendix A the whole record is one line:

```
6 7 94506455040 0 3 5 1 4 2 4 2 2 2 1 0 6 32 16 16 18 1 12 36
```

Parallel account #12. The two certificates just displayed are for the same target and are incompatible: $f = (0, 3, 4, 0, 2, 5)$ is not injective, while $f = (0, 3, 5, 1, 4, 2)$ is a permutation, and the two predecessors are different ten-cell sets. Neither can be obtained from the other by the transport of Lemma 5.5, which acts by conjugation and cannot change whether a map is injective. This is the concrete form of the fact recorded in Section 8.5: across all 10,619 shared keys, the two databases agree on no witness at all.

10.3 An explicit reaching word

The recursive constructor produces the 11-letter word in Table 7, starting at $(0, 0)$. The final letter is the certificate just displayed. The table also gives the number of occupied cells after each prefix, permitting a simple replay check. The target is valid even though its initial cell $(0, 0)$ is absent.

Table 7: A reaching word for the six-by-seven example. Image lists specify the two functions completely. Prefix cardinalities are obtained by exact replay.

Step	Row function	Column function	Cells
1	(1, 0, 2, 3, 4, 5)	(4, 1, 2, 3, 0, 5, 6)	2
2	(0, 2, 1, 3, 4, 5)	(3, 1, 2, 0, 4, 5, 6)	3
3	(0, 1, 3, 2, 4, 5)	(2, 1, 0, 3, 4, 5, 6)	4
4	(0, 1, 2, 4, 3, 5)	(1, 0, 2, 3, 4, 5, 6)	5
5	(0, 1, 2, 3, 5, 4)	(5, 1, 2, 3, 4, 0, 6)	6
6	(0, 1, 2, 3, 5, 4)	(5, 1, 2, 3, 4, 0, 6)	6
7	(0, 1, 2, 3, 4, 5)	(6, 1, 2, 3, 4, 5, 6)	7
8	(0, 1, 2, 3, 4, 5)	(0, 1, 2, 0, 4, 5, 6)	8
9	(0, 1, 2, 2, 4, 5)	(0, 1, 2, 3, 4, 5, 6)	9
10	(5, 1, 2, 3, 4, 5)	(0, 1, 2, 3, 4, 5, 6)	10
11	(0, 3, 4, 0, 2, 5)	(2, 0, 6, 5, 4, 6, 3)	14

The general length bound for this target is $14 + 6 - 2 = 18$, so this word is comfortably within the proved bound. It is not claimed to be a shortest word. The same target was also constructed and replayed from every one of its $6 \cdot 7 = 42$ possible roots.

Constructing from the B certificate instead produces a different word for the same target, of ten letters rather than eleven; it is tabulated in Appendix D and stored in `data/theorem_example_6x7_word.json`. Two constructors reading two databases give two words; neither length is a claim about optimality.

10.4 A different six-by-seven target, and a certificate no permutation search could find

Consider now the target whose ordered column codes are

$$(14, 20, 25, 26, 33, 34, 36), \quad (10.1)$$

so that, for instance, $14 = 2^1 + 2^2 + 2^3$ puts column 0 in rows 1, 2, 3. It has seventeen cells, meets all six rows and all seven columns, and is hard; its family integer is 94,591,008,768. Its certificate in database A is

$$f = (3, 5, 0, 2, 4, 5), \quad g = (2, 2, 3, 2, 0, 5, 3),$$

$$(b(M_0(T)), \dots, b(M_6(T))) = (9, 24, 16, 17, 6, 34, 10),$$

with thirteen cells and full support. The two incidence matrices are

$$S = \begin{pmatrix} 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

Parallel account #13. This example is audited in a third style, by columns rather than by source cells: write $F_j = f(M_j(T))$ for the contribution of the row branch and $G_j = \bigcup_{k:g(k)=j} M_k(T)$ for the contribution of the column branch. What that buys is an audit whose row count is the number of *columns*, seven, rather than the number of occupied source cells, thirteen — a cheaper hand check on wide targets, and one that displays which branch is responsible for each column.

Table 8: Column-wise forward-image check for the target (10.1).

j	$b(F_j)$	$b(G_j)$	Union, $b(M_j(S))$
0	12	6	14
1	20	0	20
2	16	25	25
3	24	26	26
4	33	0	33
5	32	34	34
6	36	0	36

From the root $(0, 0)$ the constructor produces a sixteen-letter word, printed in full in Appendix E and stored with every intermediate subset in `data/strip\_example\_6x7\_word.json`. Its successive subset cardinalities are

$$1, 2, 2, 3, 4, 5, 5, 6, 6, 7, 8, 9, 10, 11, 12, 13, 17.$$

The bound here is $17 + 6 - 2 = 21$.

The last of those steps is the certificate letter, and it adds *four* cells at once using noninvertible transformations. A method restricted to permutation letters, or to predecessors contained in the target, would miss this particular certificate. That observation does not claim that every certificate for this target has the property.

10.5 A residual core, certified by a bare pair of transpositions

The third target is

$$(3, 5, 9, 14, 17, 33, 50), \tag{10.2}$$

whose columns are $\{0, 1\}, \{0, 2\}, \{0, 3\}, \{1, 2, 3\}, \{0, 4\}, \{0, 5\}, \{1, 4, 5\}$: sixteen cells, all six rows used, every degree at least two, both neighborhood families incomparable, and — checked directly — no balanced triple. It is a genuine residual core.

Its certificate in database C is

$$f = (0, 1, 2, 3, 5, 4), \quad g = (0, 1, 2, 3, 5, 4, 6), \quad T = (3, 5, 9, 14, 32, 17, 50),$$

that is, f swaps rows 4, 5 and g swaps columns 4, 5, and the predecessor has fifteen cells. This is a minimal one-cell decrease, and the whole verification is two set identities:

$$f(\{5\}) \cup \{0, 4\} = \{4\} \cup \{0, 4\} = \{0, 4\}, \quad f(\{0, 4\}) \cup \{5\} = \{0, 5\} \cup \{5\} = \{0, 5\},$$

giving target columns 4 and 5; every other column is fixed pointwise or setwise, the last one $\{1, 4, 5\}$ being invariant under the row swap.

Parallel account #14. Three worked certificates, three shapes of witness: a non-injective ten-cell predecessor dropping four cells at once (Section 10.4), a permutation-row-map ten-cell predecessor (Section 10.2), and a pair of transpositions dropping one cell. What the last buys is that it is checkable by hand in two lines with no table at all, which the other two are not; what the first buys is a witness outside the reach of any permutation-restricted search.

The constructor of Section 11 reaches this target from the origin $(0, 0)$ in seventeen letters, tabulated in `residual/example\_6x7.json`; several of its two-letter isolated-edge seeds preserve cardinality on their second letter while changing positions, which is why the cardinality trace is not strictly increasing. The word is printed in Table 9.

10.6 Two opposed remarks, and their resolution

Two statements in this manuscript look contradictory and are both true.

- (i) Section 10.4: a method restricted to permutation letters, or to predecessors contained in S , would miss the certificate displayed there for the hard target (10.1).
- (ii) Lemma 8.6: *every* residual core with at most six rows has a certificate whose two components are both permutations.

The resolution is that the two statements quantify over different targets. Balanced-triple contraction (Lemma 6.2) changes which targets survive to the finite table. The target (10.1) contains a balanced triple — its columns 0, 1, 3, with neighborhoods $U = \{1, 2, 3\}$, $V = \{2, 4\}$, $W = \{1, 3, 4\}$, all three of whose pairwise unions equal $\{1, 2, 3, 4\}$ — so on the C route it never reaches the table at all; it is discharged analytically by a letter whose column component is a three-cycle. What statement (ii) asserts is a property of the residual remainder, not of every hard matrix. By contrast the target of Sections 10.1–10.2 has no balanced triple at all, so it is itself residual; its row-permutation orbit, of size 180, is covered in database C by the representative with family key 282,574,489,649,704, which therefore supplies a third and again different certificate for it, this one a pair of permutations. Conversely, database A contains no balanced-triple step, so it must certify (10.1) directly, and the witness it found is non-injective. Neither statement may be dropped in favor of the other: (i) is what forces the SMT route to exist, and (ii) is what makes the solver-free route possible.

Step	Row transformation f	Column transformation g	Subset size
1	512340	4123056	2
2	512340	4123056	2
3	102345	1023456	3
4	102345	1023456	3
5	021345	0321456	4
6	013245	0213456	5
7	013245	0213456	5
8	012435	0153426	6
9	012435	0153426	6
10	012345	0126456	7
11	012345	0133456	8
12	012345	0323456	9
13	012345	2123456	10
14	012345	1123456	11
15	012345	5123460	14
16	012345	0123656	15
17	012354	0123546	16

Table 9: The seventeen-letter reaching word for the residual core, replayed step by step. The construction is not asserted to be shortest. Several two-letter isolated-edge seeds preserve cardinality on their second letter while changing positions.

10.7 Twenty columns and a width beyond the finite classification

The unique six-row hard family with twenty columns consists of all three-element subsets of [6]. It has 60 cells. The bundled constructor reaches the corresponding ordered matrix in 35 letters; the constructor driven by database B reaches the same target in 39 letters. Both are within the proved bound of 64. Two constructors, two databases, two words: the difference measures implementation choices, not the theorem.

Repeat each of these twenty column types five times, obtaining a six-by-one-hundred matrix with 300 cells. Repeated columns are handled by the comparable-column reduction, so no hundred-column certificate is needed. The supplied reaching word has 115 letters, compared with the bound 304. Both the target and the complete word are included as JSON files. This example makes the finite-to-infinite step concrete: the finite classification is of obstruction types, not of every possible target width.

11 Constructive algorithm and independent tests

11.1 Turning the proof into a word

The supplied `construct.py` follows the induction literally. It accepts a target, dimensions, and a root. It checks validity before starting. Empty rows and columns are removed with explicit coordinate maps. A comparable-neighborhood step recursively reaches its smaller valid predecessor and appends the copying letter. An isolated-cell step prepends its one- or two-letter seed to a lifted smaller word.

For a hard target, the program tries the at most $6!$ row permutations, transforms its column codes, and locates a stored representative. The resulting column permutation is explicit because the column codes are distinct. If π, σ carry the current target to a representative with certificate (f_0, g_0, T_0) , the program uses

$$T = (\pi^{-1}, \sigma^{-1})T_0, \quad f = \pi^{-1}f_0\pi, \quad g = \sigma^{-1}g_0\sigma.$$

It checks the transported identity before recursion. On returning, it replays the entire word from the requested root and checks both the target and the length bound.

The recursion is mathematically well founded, but this reference Python implementation uses the interpreter's ordinary recursion stack. Extremely large user-supplied grids can exceed that implementation limit even though the theorem applies. No claim of an optimized production implementation is made.

11.2 Four constructors with different recursion discipline

Four reaching-word constructors are shipped. They implement the same induction and are not interchangeable in their interfaces or their checks.

Parallel account #15. `src/construct\_iterative.py` replaces the recursion by an *explicit stack* of reduction frames. What it buys is that its logical recursion depth does not impose the interpreter's recursion limit on the number of columns, so it is the engine to use on wide targets; it is the only such implementation here. Running time and memory still grow with the target, because the transformation arrays are written out explicitly.

Parallel account #16. `python/construct\_word.py` exposes an importable Constructor class with a `construct(m,n,S,start=(0,0))` method and a `-start row,column` option, and it reads database B. What it buys is programmatic reuse: the constructor can be driven from another program without a JSON round trip, and the root can be varied from the command line.

Parallel account #17. `residual/construct.py` takes `-rows, -columns, -masks, -origin` and `-output`, accepts zero masks for empty columns, raises Python's recursion limit dynamically from the target size, and checks *both* length bounds of (7.3) on every word it emits. What it buys is the only implementation that exercises the balanced-triple step and the only one that validates the dual bound.

Parallel account #18. `src/construct.py`, the constructor used in Section 10.1, takes a target as a JSON object with fields `m, n`, an optional `root`, and `target` as a list of coordinate pairs; it pre-validates the target before starting, then replays the constructed word and rechecks the length bound before writing output. What it buys is that the input and the output are both inspectable artifacts, which is exactly what the bundled examples of Section 10 are. The iterative constructor uses the same JSON shape with the key `cells` in place of `target`.

All four reject invalid targets, and all four replay the word they produce and compare the replayed result with the requested target before reporting success.

11.3 Exact breadth-first checks on small grids

A separate bitset breadth-first implementation enumerates the full transformation-pair alphabet for small grids. It does not call the word constructor’s transition routine. Its reachable-state sets agree exactly with the independently computed valid-state sets:

Grid	Reachable subsets	Largest shortest access length
2×2	10	2
2×3	44	3
3×3	400	4

These breadth-first results check small instances directly. They are corroboration, not the reason that arbitrarily large widths are covered.

11.4 Replay, coverage, and negative tests

The test suite constructed and replayed every valid target on grids 1×4 , 4×1 , 2×2 , 2×3 , 3×3 , and 3×4 . The last of these contains 3,392 valid targets. It also checked all 3,600 combinations of a valid three-by-three target and a specified root, all 42 roots of the displayed six-by-seven example, and 180 reproducibly seeded random targets on larger grids, including 6×50 and its transpose.

An additional straightforward Python antichain enumeration, using lists and set inclusion rather than the C++ bitset traversal, reproduced both total and hard-family counts for $m = 2, 3, 4, 5$.

Seven deliberately malformed certificates were rejected by the local checker. The mutations included a truncated function, an out-of-range function value, an empty predecessor, a repeated source cell, an incorrect family integer, an unsuccessful solver status, and reversed column order. A separate negative coverage test removed the sole three-row certificate. The exhaustive checker then stopped with

```
FAIL: UNCOVERED family: m=3 mask=104
```

A test suite that only accepts good data can miss an accidentally vacuous check. These rejection tests check that important failure paths are active. They do not replace code review or a formal soundness proof.

11.5 Three further regression suites

Each of the other three routes ships its own suite. The four are not duplicates: they cover different families exhaustively, and their totals are not comparable.

The iterative constructor’s suite: 111,984 replayed words. Appendix F gives the per-family breakdown. It exhausts 1×1 , 1×5 , 2×2 , 2×3 , 2×4 , 2×5 , 3×3 and 3×4 over all valid targets *and* all roots — the exhaustive 2×5 and 4×4 families appear in no other suite — adds all valid 4×4 targets at the root $(0, 0)$, relabeled hard targets with varied roots, random six-row targets up to 64 columns, and the worked example at all of its roots. A further 2,120 single-cell separation checks exercise Lemma 3.1 directly. The same runs record a reduction-usage census — 4,294 table reductions, 244,114 row copies, 130,136 column copies, 219,159 isolated-cell steps, and 176,748 empty-axis removals — which are implementation diagnostics, not additional hypotheses.

The B suite: 738 access words and 700 distinguishing words. The access-word checks exhaust 1×1 , 1×4 , 2×3 , 2×4 and 3×3 from the usual root, all valid 2×2 targets from all four roots, and 61 larger cases including randomized targets, relabeled cores with varied roots, a transposed grid, widths beyond twenty, and the maximal-width core, with seed 20260920. The 700 singleton-distinguishing-word tests are the only test suite anywhere here for the fixed three-letter construction of Section 3.4. Four named rejection scenarios were exercised against `src/verify.cpp`, with their exact failure strings: a missing record gives `exhaustive coverage failed at m=6`; a duplicated record gives `line 10622: duplicate target`; a negative integer gives `only nonnegative decimal integers are allowed`; and a corrupted witness gives `transition image is not the target`.

The C suite: 23,584 replays plus auxiliary local checks. It comprises 4,201 tests of all valid origins and targets through 3×3 ; 19,283 tests of *every certificate target at every origin*, together with transposes; and 100 deterministic pseudorandom wide targets at widths 7, 11, 20, 37, 64, twenty samples each, seed 20260920. The longest word observed was 133 letters, and both branches of (7.3) were checked on every word. The verifier additionally checked 1,050 balanced triples against Lemma 6.2, 8,100 isolated-edge seeds covering all four origin cases in grids through 6×6 , and 8,100 singleton instances of the two-letter cell test.

Encoding and coverage negatives outside the base suite. The plain-integer and mask serializations of database A were compared record by record (Section 8.2.7). Altering one row-map value to another in-range value was rejected by the single binary because the forward image no longer matched; deleting the unique three-row certificate was rejected as the uncovered key 104 — the same negative as the base suite’s, reached through a different checker.

12 Reproduction and the trust boundary

12.1 What the archive contains

The main document is `shuffle_six_state.tex` with its compiled PDF, and `tables` holds the TeX tables generated from checked data, so everything needed to rebuild the PDF is present. Four entries of the table below — `data/certificates.jsonl`, `data/certificates_masks.jsonl`, `data/certificates.txt` and `certificates/cores.cert`, about six megabytes together — are *not* distributed. They are regenerated deterministically by the routes described in this section. The 453 records of database C and every worked example cited in the text are shipped, so the displayed cases remain checkable without regenerating anything.

Path	Contents
data/certificates.jsonl	database A, coordinate-pair encoding
data/certificates_masks.jsonl	database A, column-mask encoding
data/certificates.txt	database A, plain-integer encoding
certificates/cores.cert	database B, 10,619 records
residual/certificates.tsv	database C, 453 records
data/cores.csv, core_counts.csv	B target keys and four-way counts
src/	A checkers, coverage, constructors, tests, Z3 interface
src/verify.cpp, core_enumeration.hpp	B verifier and enumeration header
src/enumerate_cores.cpp	standalone B CSV generator
python/, tests/, search/	B constructor, checks, optional finder
residual/	C: checker, generator, constructor, tests, logs
discovery/	packaged parallel regeneration driver for A
examples/	bundled targets, certificates, replayable words
audit/, audit/theorem/, logs/	recorded runs, environments, timings
SOURCE_STATUS.md	sources, attribution and novelty limitations

The three certificate files have SHA-256 digests

```
c44f2ebe73ac2584f1ff9d31cec7fc2e2560ab8fd166c80e86d80eb012c6b594
e376820a4de82e1388ea67727ee96bb0cc1c7047d38dd4ae9cfae5617c45abd4
6059c807de8aadbadfbd363c805f83109395063fbbb6e5d87cad45c8b80a48f
```

for databases A, B and C respectively. Each is one 64-character string. A digest identifies exact data; it is not a substitute for checking them, and this archive deliberately ships no checksum manifest of its own.

12.2 Verification without a solver

The checking workflow requires Python 3.10 or later and a GCC- or Clang-compatible C++17 compiler. It does not require Z3, a Python package installation, Internet access, or a proof assistant. From the archive root, run:

```
make check
```

The equivalent core commands are:

```
python3 src/check_certificates.py
c++ -O3 -std=c++17 src/coverage.cpp -o audit/coverage
./audit/coverage audit/verified_targets.tsv > audit/coverage.json
python3 src/test_suite.py
python3 src/negative_coverage_test.py
```

Each stage must exit successfully. In particular, a successful local certificate check alone does not establish coverage. Conversely, exhaustive coverage of identifiers without checking the associated witnesses does not establish the predecessor lemma.

A specific example can be regenerated using

```
python3 src/construct.py examples/six_by_seven_input.json \
--output audit/replayed_example.json
```

The supplied PDF can be rebuilt with `make pdf`, using a standard TeX installation with the packages listed in its preamble.

12.3 Three further verification paths

Each database can be checked on its own, and the three routes have deliberately different prerequisites. None of them needs Z3, a network connection, a Python package installation, or a proof assistant.

One C++ binary, database A.

```
g++ -O3 -std=c++17 -Wall -Wextra -pedantic src/verify_all.cpp -o verify_all
./verify_all data/certificates.txt
python3 src/test_artifacts.py ./verify_all
python3 src/test_construct_iterative.py
```

The final substantive line must be the PASS line of Section 8.2.6. Run the regression script *without* Python's -O flag: some of its additional test conditions use the ordinary assertion mechanism and would be removed. The constructor's own replay and certificate validation use explicit checks and are not affected by optimization flags.

One C++ binary, database B.

```
mkdir -p build
g++ -std=c++20 -O2 -Wall -Wextra -pedantic src/verify.cpp -o build/verify
./build/verify certificates/cores.cert
python3 tests/verify_certificates.py
python3 tests/test_construction.py
```

The first command both validates the supplied triples and regenerates the entire canonical representative set, comparing key sets. Expected output ends with OVERALL: PASS. The enumeration tables can be regenerated independently of the certificate file with

```
g++ -std=c++20 -O2 src/enumerate_cores.cpp -o build/enumerate_cores
./build/enumerate_cores build/cores.csv build/core_counts.csv
```

and the two CSV files compared with the shipped ones.

No compiler at all, database C.

```
cd residual
python3 verify.py --json verification.json
python3 test_construct.py
```

Python 3.9 or newer and its standard library suffice, and the expected final message is

```
PASS: certificates, exhaustive coverage, and auxiliary checks.
```

The -quick flag checks certificate soundness only, omits exhaustive coverage, and must not be represented as verification of the finite lemma. The certificate file can also be regenerated deterministically, with no solver:

```
g++ -O3 -std=c++17 generate.cpp -o generate
./generate > regenerated.tsv 2> regenerated.log
python3 verify.py --certificates regenerated.tsv
```

In the recorded run that regeneration was byte-identical to the shipped file.

Operational notes. The command-line constructors accept at most six *rows*. For a grid with at most six columns instead, transpose the target, construct the word, and then exchange the two coordinate transformations in every letter; there is no automatic transposition option. A regeneration run must be given a fresh output directory, and the A search refuses to overwrite an existing search-output file. Loading the C orbit table costs roughly a hundred megabytes of memory in this environment. On Windows, invoke the Python entry points through the PowerShell launcher rather than a shell shebang.

12.4 Optional regeneration of the search

To regenerate target representatives and search for fresh witnesses, run

```
make enumerate
python3 src/search_certificates.py
```

The search requires an installed Z3 shared library. It is located using `ctypes.util.find_library`; an explicit location can be supplied through the `Z3_LIBRARY` environment variable. The default regenerated certificate output is in `audit`, not over the bundled proof data, and the command refuses to overwrite an existing search-output file. Different solver versions may choose different witnesses. Such witnesses are acceptable precisely when they pass the same independent checks.

A ten-target regeneration smoke test was executed with the packaged search interface, and the packaged target generator exactly reproduced all 10,642 target identifiers. The full original witness-discovery run also succeeded, as recorded in the audit environment report.

A second, packaged parallel regeneration driver for the same database is in `discovery/`:

```
python3 discovery/regenerate.py regenerated --jobs 4
./verify_all regenerated/certificates.txt
```

Its three-row and four-row smoke test is recorded in `logs/regeneration_smoke.txt`. Different solver versions may find different valid predecessors; bitwise identity with the supplied database is neither expected nor required for that route. Contrast Section 12.3, where the solver-free generator of database C *was* byte-identical, because its enumeration is deterministic.

The optional finder for database B is `search/predecessor_search.py` with `search/z3local.py`; it encodes the same constraints in SMT-LIB and supports targeted rediscovery by family integer, for example `-rows 6 -family 94506455040 -timeout-ms 10000`.

12.5 Recorded environment and timing

The computations used Python 3.13.5, GCC 14.2.0, and Z3 4.13.3.0 on an x86-64 Linux environment; the database C run used Python 3.13.5 as well. Recorded elapsed times, by route:

Route	Step	Elapsed
A, two-stage	initial full witness search	224.34 s
	coverage run, peak RSS 67,412 KiB	3.55 s
	supplementary test suite	2.08 s
A, one binary	witness search after candidate enumeration	59.54 s
	original exhaustive checker run	3.28 s
	constructor tests	9.39 s
C	verification, peak RSS about 128 MiB	about 7 s

These are illustrative observations from these environments, not guaranteed runtime bounds, portable benchmarks, or a controlled comparison with previous implementations. The proof depends on the finite checks completing correctly, not on their speed. Detailed outputs are in `audit/`, `audit/theorem/`, `logs/` and `residual/`.

The file `logs/verification_ubsan.txt` records a full verification rerun under undefined-behavior sanitization. It is the only sanitizer run in this archive; it checks for undefined behavior in the C++ checker’s execution, which ordinary passing output would not reveal, and it is not a proof of the checker’s correctness.

12.6 Exactly what is, and is not, trusted

The mathematical reductions are proved in the text. The finite certificate lemma is supported by explicit witnesses, a local checker, and an independently structured exhaustive enumeration. The solver is outside this verification trust boundary: even an incorrect solver cannot make a false image equality pass a correct checker.

The Python and C++ checkers themselves, their runtimes, the compiler, and the hardware remain within the practical trust boundary. The several checkers serve different roles; none is a proof-assistant kernel. The additional Python enumeration overlaps with the C++ coverage audit only through five rows, not through six. No claim of independent human review, peer review, or fully formal verification is made.

The phrase “independent checker” throughout this manuscript means independent of the search and canonicalization algorithms — and, across the three databases, independent of one another’s representative sets and witnesses. It does not mean independent human authorship or independently maintained third-party software: every program here was written during these investigations. Separately implemented search and checking reduce correlated implementation risk; they do not eliminate it. Three databases agreeing is stronger evidence than one, but they were prepared in the same environment, and that is a residual common cause.

A fully formal version would prove the soundness and completeness of the finite checker and enumeration inside a proof assistant and import or recompute the witness table there. Until that is done, the accurate classification is a reproducible computer-assisted proof, not a kernel-checked theorem.

13 What remains open and how to extend the attack

13.1 The unrestricted conjecture

The argument establishes the strip $\min(m, n) \leq 6$ and does not establish $\mathcal{P}_h$ for any $h \geq 7$. Therefore it does not settle the case in which both operands have at least seven states. Positive computational results in random larger matrices, even if numerous, would not justify that extrapolation.

The obstruction reduction survives unchanged at seven rows, where the width bound becomes $\binom{7}{3} = 35$. However, even the subfamilies of just the three-element subsets already form 2^{35} different antichains. This elementary lower bound explains why extending the same labeled coverage traversal without further compression is a materially larger computation. It is not an argument that the next case is inaccessible.

Two concrete routes are suggested by the present certificates. One is to classify certificates by structural templates, reducing whole classes of antichains symbolically. Another is to use a verified isomorph-free construction in the *coverage* checker itself, while retaining a separate completeness proof. Either route must preserve the root-validity invariant that made the current unrooted classification sound. A third is to look for further analytic reductions in the style of Lemma 6.2: that one lemma alone cut the six-row obligation from 5,237,599 labeled families to 168,673, and nothing in its proof mentions six.

13.2 Three formulations of the open step

The unresolved step of this approach can be stated as a conjecture, as a question, or as a proved conditional. These are three different objects and all three are recorded.

13.2.1 As a conjecture

Conjecture 13.1 (Spanning predecessor conjecture). *Every hard finite zero-one matrix has a spanning predecessor certificate in the sense of Definition 5.4.*

The present computation proves this assertion for at most six rows or at most six columns. Theorem 7.2 shows that the general auxiliary conjecture would imply the full shuffle state-complexity conjecture, together with the same access-word bound.

The converse is not established. A reachable hard target might have no smaller *spanning* predecessor and still be reachable from a smaller merely valid predecessor, or by a path whose final useful step does not reduce cardinality. Consequently a counterexample to Conjecture 13.1 would refute this particular one-step strategy, not automatically the original shuffle conjecture. A solver returning “unknown” refutes neither statement.

13.2.2 As a question, with a combinatorial reformulation

Parallel account #19. Stated as a question rather than a conjecture, the same step invites a reformulation that the conjecture form does not: fix the letter first. What that buys is an exact finite-combinatorial restatement of the *existence* problem for a given pair of maps, which is where a structural proof would have to start.

Question 13.2. Does every hard target, in arbitrary finite dimensions, admit a strictly smaller full-support predecessor?

Fix f and g first. A candidate source cell (i, j) is allowed exactly when both $(f(i), j)$ and $(i, g(j))$ lie in S ; regard its two images as an edge on the vertex set S , a loop when they coincide. Selecting source cells is then covering all vertices of S by these edges, subject to the extra requirement that every row and every column be represented among the *sources*. This is an exact reformulation for fixed maps, and Proposition 5.7 solves the unconstrained version of it exactly. The hard part is choosing the maps so that a full-support cover uses fewer than $|S|$ source cells. This is an observation about the certificate equations, not a solved structural theorem.

13.2.3 As a proved conditional

Parallel account #20. The third formulation is not a conjecture at all but a theorem: the implication itself, stated in two strengths. What it buys over the conjecture form is that it is *proved*, so it can be cited; and that it isolates the permutation-pair case, which is what a solver-free search would actually test.

Proposition 13.3. *If every residual core in arbitrary dimensions has a strictly smaller full-support predecessor under some letter of the full transformation alphabet, then every valid grid subset in arbitrary dimensions is reachable, and the bounds of Section 7 hold there. If the predecessor letter can always be chosen to be a pair of permutations, the same conclusion follows as a special case.*

Proof. Repeat the induction of Theorem 7.2, substituting the hypothesized statement for $\mathcal{R}_h$ and invoking Theorem 7.4; every other reduction, including Lemma 6.2, is already dimension-independent. Then apply the distinguishability argument of Section 3. $\square$

A failed permutation search in a larger dimension would admit three readings, which must be distinguished. It could refute the permutation-pair form while leaving a predecessor under a nonpermutation letter — exactly the situation of Section 10.6. Even failure of *every* one-step strict predecessor would not prove nonreachability, since a reaching path could arrive through an intermediate subset of the same or larger cardinality. A genuine counterexample to the shuffle conjecture requires proving the absence of *all* reaching words.

The most concrete continuations are therefore a support-constrained version of the edge-cover problem above, or a structural classification of balanced-triple-free double-antichain incidence systems. Both are sharply stated finite-combinatorial questions rather than searches through a powerset automaton, and neither is claimed to be resolved beyond six rows.

13.3 Alphabet size and the one-state exception

All constructive words use the full alphabet $\Gamma_{m,n}$. The result is not a small-alphabet witness construction. It also does not imply that an arbitrary pair of m - and n -state automata has maximal shuffle complexity: special transition structures can yield much smaller languages.

The assumption $m, n \geq 2$ in the state-complexity theorem is essential to the proof of distinguishability. That proof requires nonfinal states in both components. The one-dimensional reachability construction remains valid, but different reachable subsets can then be equivalent as language states. Reachability of every valid subset and attainment of their total number are separate assertions; they must not be conflated.

It is also not proved here that every valid target has a strictly smaller one-letter predecessor. The induction deliberately keeps the empty-axis and isolated-cell reductions separate: an isolated-cell seed may need two letters and may pass through a subset of equal cardinality. The verified one-letter property concerns only the hard class, and there only with full-support predecessors.

13.4 A connection with dependent proof objects

The artifacts naturally separate discovery from proof checking. In a dependent-type presentation, a certificate for a fixed target S can be represented schematically as

$$\text{Cert}(S) = \sum_{f:R \rightarrow R} \sum_{g:C \rightarrow C} \sum_{T:\mathcal{P}(R \times C)} (\text{Span}(T) \times (|T| < |S|) \times (\Delta_{f,g}(T) = S)).$$

Here $\text{Span}(T)$ means full row and column support, and the last components are proofs rather than unchecked flags. The finite computation supplies concrete candidates for the first three components. A verified decidable checker would supply the remaining proof components. Well-founded recursion on $(m + n, |S|)$ then turns these local proof objects into reaching words with correctness proofs.

This gives a precise formalization target rather than an unverified code sketch. The nontrivial formal obligations are the enumeration's completeness, transport of certificates, root-sensitive reductions, and termination of the constructor. None is delegated to the mere presence of a solver-generated file.

Two further presentations of the same interface are worth recording. The first names the certificate type before any checking has occurred,

$$\text{PreCert}(S) = \sum_{f:[m] \rightarrow [m]} \sum_{g:[n] \rightarrow [n]} \{T \subseteq [m] \times [n] : \text{Span}(T) \wedge |T| < |S| \wedge \Delta_{f,g}(T) = S\},$$

so that a search *proposes* the computational components f, g, T and a checker supplies the finite proof obligations that make the proposal inhabit the specification. The second makes the obligations explicit as named components of a tuple,

$$(T, f, g; h_{\text{size}}, h_{\text{row}}, h_{\text{col}}, h_{\text{image}}),$$

asserting respectively $|T| < |S|$, $\text{rows}(T) = [m]$, $\text{cols}(T) = [n]$, and $\Delta_{f,g}(T) = S$; a permutation certificate carries two further components proving bijectivity of f and of g . A small kernel would then need two theorems and no more: that the certificate-checking Boolean function implies those properties, and that the recursive enumeration covers every irreducible target exactly once — for database C the latter is the hereditary-pruning argument of Remark 6.4.

The logical shape of the whole result is

$$\text{verified finite coverage} \implies \forall n \geq 1 \forall S \subseteq [6] \times [n] : \text{Valid}(S) \implies \exists w \Delta_w(\{(r, c)\}) = S.$$

The universal quantifier over n is discharged by the mathematical induction, not by evaluating a finite list of column sizes. Root independence comes from full support, so no hidden quotient by initial-state-changing permutations is required. Importing only the count 10,619, or 10,642, or 453 as an axiom would not accomplish the coverage proof, which is the part that matters.

13.5 Four tempting shortcuts that would be invalid

Each of the following would look like a proof of Theorem 1.1 and would not be one; each is addressed separately above.

1. Numerical agreement with $F(m, n)$ for finitely many pairs (m, n) says nothing about all n .
2. Counting canonical configurations without preserving the initial-state requirement does not establish reachability: transport moves the root (Section 5).
3. A satisfiable solver log without witnesses is a weaker artifact than a checkable certificate file, and is not accepted here at any point.
4. Reachability without distinguishability is not a minimal-DFA lower bound (Section 3).

14 Conclusion

The selected problem was an exact state-complexity conjecture for the shuffle of regular languages. The outcome is a computer-assisted proof for the entire six-state strip, supported by finite tables of local witnesses and exhaustive independent coverage, rather than by a sample of successful automata calculations.

The essential compression is mathematical: remove empty dimensions, contained neighborhoods, and isolated cells; optionally contract balanced triples; classify the remaining antichains; require predecessors to have full support; and induct on size. The same proof yields explicit reaching words, of length at most $|S| + \min(m, n) - 2$, at most $|S| + \min(r_S, c_S) - 2$ in the support form, and at most $2(|S| - 1)$ by the dual estimate.

What this merged archive adds to any one of its sources is redundancy of the right kind. The finite premise is discharged three times, over two different classifications of the irreducible remainder, by two different canonicalization correctness proofs, from two different discovery engines — one of which needs no solver — and audited by four checkers with different prerequisites, down to one that needs no compiler. Not one witness is shared between the two databases that classify the same objects. The unrestricted conjecture remains beyond the result proved here, while the certificate data, the auxiliary conjecture, its question form, and its proved conditional form provide concrete next objects for further study.

A Certificate schemas and minimal verification cores

Three serialization families are in active use by shipped checkers. They are not interchangeable, and each is read by a different program.

A.1 Coordinate-pair JSON lines, and its plain-integer twin

Each line of `data/certificates.jsonl` is an independent JSON object. The fields `m`, `n`, and `family` give the dimensions and family integer. The list `columns` contains the strictly increasing column codes. Lists `f` and `g` are function image tables. The list `T` consists of distinct pairs of zero-based coordinates. The field `status` records that a positive witness was found; its text is not itself evidence of correctness.

After structural validation, the mathematical core of a local check can be written as follows. The executable checker contains the additional range, antichain, degree, and schema checks described in the text.

```
S = {(i, j)
      for j, column in enumerate(record["columns"])
      for i in range(m)
      if column & (1 << i)}
T = {tuple(cell) for cell in record["T"]}
f, g = record["f"], record["g"]

require(len(T) == len(record["T"]), "duplicate cell")
require({i for i, j in T} == set(range(m)), "missing row")
require({j for i, j in T} == set(range(n)), "missing column")
require(len(T) < len(S), "not a smaller predecessor")
image = {(f[i], j) for i, j in T} | {(i, g[j]) for i, j in T}
require(image == S, "incorrect transition image")
```

The amount of arithmetic needed to check an individual witness is tiny: each source cell produces exactly two image cells. The serious global obligation is not checking one witness, but showing that no irreducible isomorphism type was omitted. This is the role of the second checker.

The same records appear in `data/certificates_masks.jsonl` with the predecessor given as a list of column bit masks under the key `predecessor` and the family integer under the key `key`; and in `data/certificates.txt`, a header-commented file with one whitespace-separated record per line,

```
m n family_key f[0] ... f[m-1] g[0] ... g[n-1] P[0] ... P[n-1]
```

where $P[j]$ is the mask of the predecessor's j th column. This is the form read by `src/verify_all.cpp`, which works entirely in column masks: for every bit i of $P[j]$ it accumulates

$$\text{image}[j] \mid= 2^{f(i)}, \quad \text{image}[g(j)] \mid= 2^i,$$

and then compares the resulting array with the target array, with no search or symbolic inference. Section 8.2.7 describes the agreement test that exists only because both forms are shipped.

A.2 Fixed-width .cert records for database B

Each noncomment line of `certificates/cores.cert` contains exactly $3 + m + 2n$ decimal nonnegative integers,

```
m n family f[0] ... f[m-1] g[0] ... g[n-1] t[0] ... t[n-1]
```

where `family` is (8.1), whose set-bit positions in increasing order are the target column codes, and the t_j are predecessor column masks. Comment lines begin with `##`, and the file carries the magic header line `## SHUFFLE-FULL-SUPPORT-1`. For example, the unique three-row hard matrix has columns 3, 5, 6 and family integer 104. The C++ checker validates input domains before shifting bits or indexing arrays; the additional permutation check on f is specific to this database.

A.3 Bar-separated records for database C

Each record of `residual/certificates.tsv` has five fields separated by a literal vertical bar,

```
m n family_key | target_columns | row_permutation | column_permutation | predecessor_columns
```

with whitespace-separated integer vectors inside each field. Despite the `.tsv` suffix the separator is a bar, not a tab. The predecessor columns are listed in the representative's original column order and *must not* be sorted independently; sorting them would in general destroy the certificate. Both f and g are permutations in every record.

B A review checklist

The following questions isolate the places where a false proof of this kind could otherwise appear convincing.

1. Are the component functions total and in range? Does the chosen alphabet contain every letter used in the proof?
2. Does each predecessor remain valid for the root? Are equal neighborhoods and mixed cases of the isolated-cell reduction handled correctly?
3. Does the hard-target classification forget only symmetries under which the *certificate property*, not merely a guessed reachability property, is invariant?
4. Are both inclusions in the image equality checked, along with strict cardinality reduction and all support requirements?
5. Does coverage include every antichain, not just maximal antichains or a list of representatives produced by an unverified canonicalizer?
6. Does the state-complexity conclusion include distinguishability and minimality of the operand automata, rather than just a reachable-state count?
7. Are the finite computation, the infinite induction, the provisional novelty assessment, and the absence of formal kernel verification clearly separated?

Each of these obligations is addressed explicitly in the proof and the provided audit workflow. Independent reviewers are encouraged to begin with these points rather than with the solver's search strategy. In this merged archive, item 5 should be checked separately for each of the three databases, since they answer it by three different arguments (Sections 8.2.5, 8.3 and 8.4).

C Six-row antichains by number of columns

Table 10 is taken from the actual exhaustive checker output, not from an external enumeration table. Degenerate antichains, including the empty family and the singleton families, are counted in the middle column even though they are not hard. The middle column is the denominator the coverage traversal actually visits; the right-hand column is the set of families that must be certified.

Table 10: Complete six-row distribution.

Columns n	All labeled antichains	Hard labeled antichains
0	1	0
1	64	0
2	1,351	0
3	14,000	0
4	82,115	30
5	304,752	5,100
6	759,457	117,821
7	1,308,270	564,330
8	1,613,250	1,113,390
9	1,484,230	1,264,720
10	1,067,771	998,987
11	635,044	618,712
12	326,990	323,900
13	147,440	147,020
14	57,675	57,645
15	19,238	19,238
16	5,325	5,325
17	1,170	1,170
18	190	190
19	20	20
20	1	1
Total	7,828,354	5,237,599

D The ten-letter word from database B

This is the word of Section 10.2 for the target with column codes (12, 17, 18, 24, 33, 34, 36), constructed from the database B certificate. Each tuple gives the images of the states in increasing order, and the letters act by (2.2). The initial subset is $\{(0, 0)\}$. The column $|S_k|$ is a diagnostic only: matching cardinalities alone would not establish that the final set is correct.

k	Row map f_k	Column map g_k	$ S_k $
1	(1, 0, 2, 3, 4, 5)	(4, 1, 2, 3, 0, 5, 6)	2
2	(0, 2, 1, 3, 4, 5)	(3, 1, 2, 0, 4, 5, 6)	3
3	(0, 1, 3, 2, 4, 5)	(6, 1, 2, 3, 4, 5, 0)	4
4	(0, 1, 2, 4, 3, 5)	(5, 1, 2, 3, 4, 0, 6)	5
5	(0, 1, 2, 3, 5, 4)	(1, 0, 2, 3, 4, 5, 6)	6
6	(0, 1, 2, 3, 4, 5)	(0, 2, 2, 3, 4, 5, 6)	7
7	(0, 1, 5, 3, 4, 5)	(0, 1, 2, 3, 4, 5, 6)	8
8	(0, 1, 2, 2, 4, 5)	(0, 1, 2, 3, 4, 5, 6)	9
9	(0, 4, 2, 3, 4, 5)	(0, 1, 2, 3, 4, 5, 6)	10
10	(0, 3, 5, 1, 4, 2)	(4, 2, 2, 2, 1, 0, 6)	14

The same data in machine-readable form is `data/theorem_example_6x7_word.json`. For an independent direct check, start from the singleton, apply the two image operations at every step, and compare the final seven column codes.

E The sixteen-letter word for the second six-by-seven target

Each f -tuple lists the images of rows 0 through 5 and each g -tuple lists the images of columns 0 through 6; the letters are read from top to bottom. The final column gives the number of occupied cells after that letter, starting from the singleton $\{(0, 0)\}$. The exact subsets, not only their sizes, are stored in `data/strip_example_6x7_word.json` and are replayed by the constructor.

Table 11: A sixteen-letter reaching word for the target (10.1).

Step	f	g	Cells
1	(1, 0, 2, 3, 4, 5)	(1, 0, 2, 3, 4, 5, 6)	2
2	(1, 0, 2, 3, 4, 5)	(1, 0, 2, 3, 4, 5, 6)	2
3	(0, 2, 1, 3, 4, 5)	(0, 6, 2, 3, 4, 5, 1)	3
4	(0, 1, 3, 2, 4, 5)	(0, 4, 2, 3, 1, 5, 6)	4
5	(0, 1, 2, 4, 3, 5)	(0, 2, 1, 3, 4, 5, 6)	5
6	(0, 1, 2, 4, 3, 5)	(0, 2, 1, 3, 4, 5, 6)	5
7	(0, 1, 2, 3, 5, 4)	(0, 1, 5, 3, 4, 2, 6)	6
8	(0, 1, 2, 3, 5, 4)	(0, 1, 5, 3, 4, 2, 6)	6
9	(0, 1, 2, 3, 4, 5)	(0, 1, 3, 3, 4, 5, 6)	7
10	(0, 1, 2, 3, 4, 5)	(3, 1, 2, 3, 4, 5, 6)	8
11	(0, 1, 2, 3, 4, 5)	(0, 0, 2, 3, 4, 5, 6)	9
12	(0, 1, 2, 3, 4, 5)	(0, 1, 1, 3, 4, 5, 6)	10
13	(0, 3, 2, 3, 4, 5)	(0, 1, 2, 3, 4, 5, 6)	11
14	(0, 1, 2, 3, 4, 1)	(0, 1, 2, 3, 4, 5, 6)	12
15	(0, 1, 1, 3, 4, 5)	(0, 1, 2, 3, 4, 5, 6)	13
16	(3, 5, 0, 2, 4, 5)	(2, 2, 3, 2, 0, 5, 3)	17

F Detailed regression-test counts

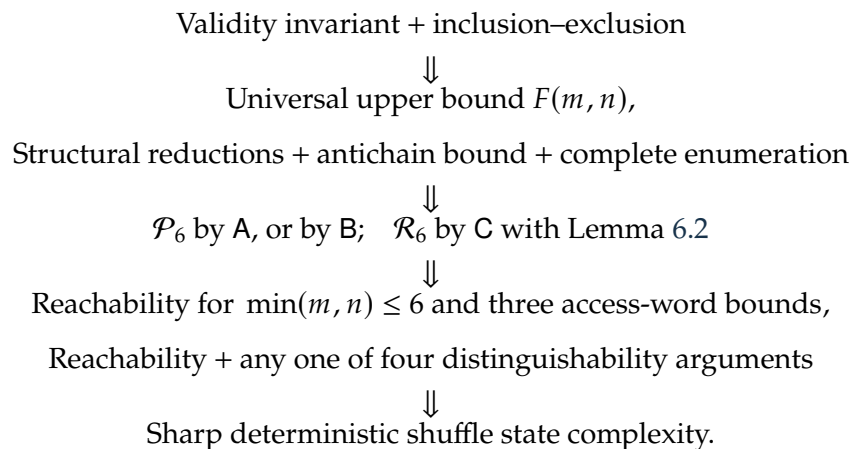
The following counts describe the executed deterministic suite of Section 11.5. Tests with different roots are counted separately, since validity and reaching words depend on the distinguished axes.

Test family	Replayed words
1×1 , all valid targets and roots	1
1×5 , all valid targets and roots	80
2×2 , all valid targets and roots	40
2×3 , all valid targets and roots	264
2×4 , all valid targets and roots	1,472
2×5 , all valid targets and roots	7,520
3×3 , all valid targets and roots	3,600
3×4 , all valid targets and roots	40,704
4×4 , every valid target, root $(0, 0)$	57,856
Relabeled hard targets, varied roots	240
Random six-row targets, up to 64 columns	75
Matching targets and all roots of the worked example	132
Total	111,984

The random generator is seeded by the integer 20260920.

G Proof dependency summary

The logical flow of the argument is:



The prior results for at most five states and for 6×6 position the work historically; they are not used as unverified computational assumptions anywhere in this chain.

References

- [1] Janusz Brzozowski, Galina Jirásková, Bo Liu, Aayush Rajasekaran, and Marek Szykuła. On the state complexity of the shuffle of regular languages. In *Descriptive Complexity of Formal*

- Systems (DCFS 2016)*, Lecture Notes in Computer Science 9777, pages 73–86. Springer, 2016. DOI: [10.1007/978-3-319-41114-9_6](https://doi.org/10.1007/978-3-319-41114-9_6). Accessible version: [arXiv:1512.01187v3](https://arxiv.org/abs/1512.01187v3), 15 July 2016.
- [2] Pascal Caron, Jean-Gabriel Luque, and Bruno Patrou. A combinatorial approach for the state complexity of the shuffle product. *Journal of Automata, Languages and Combinatorics*, 25(4):291–320, 2020. DOI: [10.25596/jalc-2020-291](https://doi.org/10.25596/jalc-2020-291). Accessible preprint: [arXiv:1905.08120](https://arxiv.org/abs/1905.08120), 20 May 2019. The preprint, rather than the access-restricted journal full text, was used for the mathematical source review.
- [3] Emanuel Sperner. Ein Satz über Untermengen einer endlichen Menge. *Mathematische Zeitschrift*, 27:544–548, 1928. DOI: [10.1007/BF01171114](https://doi.org/10.1007/BF01171114).
- [4] The Z3 contributors. Z3 C API documentation. https://z3prover.github.io/api/html/group__capi.html. Consulted 20 September 2026. The computation used the locally installed shared library reporting version 4.13.3.0.